

HMS Core - Detailed Use Cases Documentation

Student Module - Hostel Search & Discovery

USE CASE 1: Search Hostels Using GPS Location

Element: Detail

Name: Search Hostels Near Current Location

ID: UC-STU-001

Description: The student wants to find hostels near their current geographical location without manually entering an address. The system uses GPS to detect location and displays nearby hostels within a configurable radius.

Actors: Student (Primary), GPS System (Secondary), Backend Server (Secondary)

Organization Benefits:

- Reduces search friction and improves user experience
- Increases engagement by providing instant relevant results
- Reduces bounce rate from complicated search processes
- Helps students discover hostels they might not know about

Preconditions:

- Student must have the app installed and opened
- Device GPS must be enabled
- Student must grant location permission to the app
- Device must have internet connectivity
- Student must be in a location where hostel services are available

Triggers:

- Student taps "Near Me" button on home screen
- Student taps location icon in search bar
- App auto-suggests "Search nearby hostels" on first launch

Main Course:

1. Step 1: Student opens the HMS Core app
2. Step 2: System displays home screen with search options
3. Step 3: Student taps "Near Me" or location icon button
4. Step 4: System requests location permission (if not granted previously)
5. Step 5: Student grants permission
6. Step 6: System accesses device GPS and retrieves coordinates (latitude, longitude)
7. Step 7: System sends coordinates to backend server
8. Step 8: Backend calculates distance from each hostel in database to student's location
9. Step 9: Backend applies default filters: 5km radius, all available hostels
10. Step 10: System retrieves and returns sorted list of hostels (nearest first)
11. Step 11: Frontend displays results in list view with distance shown for each hostel
12. Step 12: Student can view hostel cards with: name, image, price, distance, rating, available beds
13. Step 13: Student can tap on any hostel to view details or apply additional filters

Alternate Course:

AC1: Location Permission Denied

- Condition: Student denies location permission
1. Step 1: System shows message: "Location access required for nearby search"
 2. Step 2: System provides "Settings" button to enable permission
 3. Step 3: System offers alternative: "Or search by university/area name"
 4. Step 4: Student can manually enter location or university

AC2: GPS Not Available

- Condition: Device GPS is turned off or not functioning
1. Step 1: System detects GPS unavailable
 2. Step 2: System prompts: "Turn on location services"
 3. Step 3: System provides "Settings" shortcut
 4. Step 4: System falls back to IP-based approximate location
 5. Step 5: System warns: "Using approximate location. Results may be less accurate"

AC3: No Hostels Found in Radius

- Condition: No hostels within default 5km radius
1. Step 1: System shows "No hostels found within 5km"
 2. Step 2: System automatically increases radius to 10km
 3. Step 3: System displays message: "Showing hostels within 10km"
 4. Step 4: System provides radius adjustment slider
 5. Step 5: Student can manually adjust radius up to 20km

Exception Courses:

EX1: Network Connection Lost During Search

- Condition: Internet disconnects while fetching results
- 1. Step 1: System detects network failure
- 2. Step 2: System shows cached recent search results (if available)
- 3. Step 3: System displays offline banner: "No internet. Showing cached results"
- 4. Step 4: System enables offline browsing of cached hostels
- 5. Step 5: System retries automatically when connection restored

EX2: GPS Coordinates Inaccurate

- Condition: GPS returns incorrect location
- 1. Step 1: System displays results but location seems wrong
- 2. Step 2: Student taps "Location seems wrong?"
- 3. Step 3: System refreshes GPS coordinates
- 4. Step 4: System allows manual location correction
- 5. Step 5: System re-runs search with corrected location

EX3: Backend Server Error

- Condition: Server fails to process request
- 1. Step 1: System receives 500 error from backend
- 2. Step 2: System shows user-friendly message: "Something went wrong. Please try again"
- 3. Step 3: System logs error for debugging
- 4. Step 4: System provides "Retry" button
- 5. Step 5: System falls back to cached data if available

Postconditions:

Success:

- Student sees list of hostels sorted by distance
- Each hostel shows accurate distance from current location
- Student's search is saved in recent searches
- Location preference is saved for future searches
- Analytics event logged: "nearby_search_completed"

Failure:

- Student is guided to alternative search methods
 - Error is logged for system improvement
 - Student can still access other app features
-

USE CASE 2: Search Hostels by University Name

Element: Detail

Name: Search Hostels Near Specific University

ID: UC-STU-002

Description: The student wants to find hostels near their university or college by entering the institution's name. The system provides autocomplete suggestions, retrieves the university's coordinates, and displays hostels within a configurable radius from the campus.

Actors: Student (Primary), Autocomplete Service (Secondary), Geocoding Service (Secondary), Backend Server (Secondary)

Organization Benefits:

- Targets students who know their university but are unfamiliar with the area
- Simplifies search for new students relocating for education
- Increases conversion by showing most relevant hostels for specific universities
- Builds university-specific communities within hostels
- Enables targeted marketing to specific university students

Preconditions:

- Student must have app installed and opened
- University must exist in system database
- University geolocation data must be available
- Device must have internet connectivity
- Student must know at least partial university name

Triggers:

- Student taps on search bar on home screen
- Student selects "Search by University" option
- Student starts typing university name in search field
- Student taps on a popular university card/chip displayed on home screen

Main Course:

1. Step 1: Student opens HMS Core app and navigates to home screen
2. Step 2: Student taps on main search bar
3. Step 3: System displays keyboard and search input field with placeholder: "Enter university name"
4. Step 4: Student begins typing university name (e.g., "Delhi Uni...")
5. Step 5: System triggers autocomplete after 2+ characters typed

6. Step 6: Autocomplete service queries university database with partial name
7. Step 7: System returns dropdown list of matching universities with:
 - Full university name
 - University logo/icon
 - Location (city, state)
 - Common aliases/short names
8. Step 8: Student selects desired university from dropdown
9. Step 9: System retrieves university's stored coordinates (latitude, longitude) from database
10. Step 10: System applies default search parameters:
 - Radius: 5km from university main gate/center
 - Status: Available hostels only
 - Sort: Recommended (AI-based)
11. Step 11: Backend server calculates distances from university to all hostels
12. Step 12: Backend filters hostels within radius and retrieves details
13. Step 13: AI algorithm ranks hostels based on:
 - Distance from university
 - Popularity among that university's students
 - Price appropriateness for area
 - Ratings and reviews
14. Step 14: System displays results screen with:
 - Search header: "Hostels near [University Name]"
 - Result count: "47 hostels found"
 - Map centered on university
 - List of hostel cards with distance shown
15. Step 15: Student can scroll through results, view on map, or apply filters
16. Step 16: System saves university search to recent searches

Alternate Course:

AC1: University Not Found in Database

- Condition: Student types university name that doesn't exist in database
1. Step 1: Autocomplete returns no results
 2. Step 2: System shows message: "University not found"
 3. Step 3: System suggests: "Try searching by city/area instead"
 4. Step 4: System provides "Request University Addition" button
 5. Step 5: Student can submit university details for admin to add
 6. Step 6: System logs missing university for database update

AC2: Multiple Campus Locations

- Condition: University has multiple campuses in different locations
1. Step 1: Student selects university from dropdown
 2. Step 2: System detects multiple campus locations in database

3. Step 3: System displays sub-menu: "Which campus?"
 - North Campus
 - South Campus
 - East Campus
4. Step 4: Student selects specific campus
5. Step 5: System uses selected campus coordinates for search
6. Step 6: Search proceeds with campus-specific location

AC3: Popular University Quick Selection

- Condition: Student's university appears in popular/trending list
- 1. Step 1: Home screen displays "Popular Universities" section with cards
- 2. Step 2: Student taps directly on university card
- 3. Step 3: System bypasses autocomplete, directly uses university coordinates
- 4. Step 4: Search executes immediately
- 5. Step 5: Faster result display (no typing required)

AC4: Refine Radius Around University

- Condition: Default 5km shows too many/too few results
- 1. Step 1: Results displayed with default 5km radius
- 2. Step 2: Student adjusts radius slider (1-20km)
- 3. Step 3: System dynamically updates result count: "32 hostels within 3km"
- 4. Step 4: Student releases slider
- 5. Step 5: System re-queries backend with new radius
- 6. Step 6: Results refresh automatically
- 7. Step 7: Map view adjusts zoom to show new radius circle

Exception Courses:

EX1: Geocoding Service Failure

- Condition: University coordinates cannot be retrieved
- 1. Step 1: System attempts to fetch coordinates
- 2. Step 2: Geocoding service times out or returns error
- 3. Step 3: System falls back to city-level search
- 4. Step 4: System shows warning: "Using approximate location for [University]"
- 5. Step 5: Results may be less accurate but still displayed
- 6. Step 6: System logs error for admin review

EX2: No Hostels Available Near University

- Condition: Zero hostels within 20km radius
- 1. Step 1: Search executes but returns 0 results
- 2. Step 2: System shows: "No hostels found near [University Name]"
- 3. Step 3: System suggests:

- "Search in nearby cities?"
- "Set up alert when hostels become available"
- 4. Step 4: Student can create availability alert
- 5. Step 5: System notifies student when hostel is listed near that university

EX3: Autocomplete Service Down

- Condition: Autocomplete API fails
- 1. Step 1: Student types but no suggestions appear
- 2. Step 2: System detects autocomplete failure
- 3. Step 3: System allows student to type complete name and press "Search"
- 4. Step 4: System performs fuzzy matching on entered text
- 5. Step 5: System displays best-match universities
- 6. Step 6: Student confirms correct university

EX4: Slow Network During Search

- Condition: Poor internet causes delay in loading results
- 1. Step 1: Student selects university and search begins
- 2. Step 2: Network is slow, results take >5 seconds
- 3. Step 3: System displays loading skeleton screens
- 4. Step 4: System shows progress indicator: "Finding hostels..."
- 5. Step 5: System loads results progressively as they arrive
- 6. Step 6: Student can start viewing first results while rest load

Postconditions:

Success:

- Student sees hostels near selected university sorted by relevance
- Each hostel card shows distance from university
- University search is saved to student's profile for quick access
- Search saved in recent searches with university name
- AI learns student's university preference for future recommendations
- Analytics logged: "university_search_completed" with university ID
- System tracks popular universities for homepage trending section

Failure:

- Student is guided to alternative search methods
- Error logged for system debugging
- Student can retry search or use different search method
- Recent searches still accessible

USE CASE 3: Apply Basic Price Filter

Element: Detail

Name: Filter Hostels by Price Range

ID: UC-STU-003

Description: The student wants to narrow down hostel search results based on their monthly budget. The system provides a dual-range slider and preset price brackets to filter hostels that match the student's financial capacity. Results update in real-time as the filter is adjusted.

Actors: Student (Primary), Filter Service (Secondary), Backend Server (Secondary)

Organization Benefits:

- Reduces decision fatigue by showing only affordable options
- Increases conversion by matching students with budget-appropriate hostels
- Reduces time spent browsing irrelevant listings
- Provides data on student budget patterns for market analysis
- Helps hostel owners price competitively based on demand

Preconditions:

- Student must have performed initial search (by location/university/area)
- Search results must be displayed on screen
- At least 5+ hostels must be in initial results for filtering to be meaningful
- Price data must be available for all hostels in results
- Device must have internet connectivity for real-time filtering

Triggers:

- Student taps "Filters" button on search results screen
- Student taps "Price" filter chip/option
- Student taps a preset price range chip (e.g., "Under ₹5000")
- AI suggests: "Too many results? Try filtering by price"

Main Course:

1. Step 1: Student is viewing search results (e.g., 47 hostels near Delhi University)
2. Step 2: Student taps "Filters" button (typically top-right or floating button)
3. Step 3: System opens filter panel/sheet (slides up from bottom or from side)
4. Step 4: Filter panel displays all available filter categories with "Price" at the top
5. Step 5: Price section shows:
 - Current price range: ₹0 - ₹50,000 (maximum in results)
 - Dual-handle slider for min and max price
 - Preset chips: "Under ₹5k", "₹5-10k", "₹10-15k", "₹15-20k", "Above ₹20k"
 - Toggle: "Include food in price" (shows all-inclusive vs rent-only)

6. Step 6: Student either:
 - Option A: Taps a preset chip (e.g., "₹5-10k")
 - Option B: Manually drags slider handles to set custom range
7. Step 7: If Option A (preset):
 - System automatically sets slider to ₹5,000 - ₹10,000
 - Preset chip highlights/activates
8. Step 8: If Option B (manual):
 - Student drags minimum handle to ₹6,000
 - Student drags maximum handle to ₹12,000
 - Selected range displays above slider: "₹6,000 - ₹12,000"
9. Step 9: As slider moves, system shows real-time result count:
 - "23 hostels in this range"
 - Updates dynamically as handles move
10. Step 10: System sends filter parameters to backend:
 - min_price: 6000
 - max_price: 12000
 - include_food: true/false
11. Step 11: Backend filters hostel list:
 - If "Include food" is OFF: filters by base rent only
 - If "Include food" is ON: filters by total monthly cost (rent + food)
12. Step 12: Backend returns filtered results (23 hostels)
13. Step 13: System automatically closes filter panel OR shows "Apply" button
14. Step 14: Student taps "Apply" (if button present) or filter auto-applies
15. Step 15: Search results screen updates:
 - Shows only 23 hostels within ₹6,000-₹12,000 range
 - Active filter chip appears: "₹6k-₹12k X" (with remove option)
 - Header updates: "23 hostels near Delhi University"
16. Step 16: Results remain sorted by previous sort option (Recommended, Nearest, etc.)
17. Step 17: Student can browse filtered results or add more filters

Alternate Course:

AC1: Overly Restrictive Price Range

- Condition: Student sets range that results in 0-2 hostels
1. Step 1: Student sets price range to ₹3,000-₹4,000
 2. Step 2: Real-time counter shows: "Only 1 hostel in this range"
 3. Step 3: System displays warning: "Very limited options. Consider adjusting price."
 4. Step 4: System suggests: "Try ₹3,000-₹5,000 for 8 more options"
 5. Step 5: Student can tap suggestion to auto-adjust slider
 6. Step 6: OR student proceeds with current range despite warning

AC2: No Hostels Match Price Range

- Condition: Selected range has zero matching hostels

1. Step 1: Student sets price range ₹2,000-₹3,000
2. Step 2: System shows: "0 hostels in this range"
3. Step 3: System displays message on applying: "No hostels match your price range"
4. Step 4: System shows alternatives:
 - "Nearest price: ₹4,500/month" (shows cheapest available)
 - "Adjust to ₹3,000-₹5,000 for 12 hostels"
5. Step 5: System provides quick-adjust buttons
6. Step 6: Student can widen range or remove filter

AC3: Toggle "Include Food" Changes Results

- Condition: Student switches between rent-only and all-inclusive pricing
1. Step 1: Student has price filter ₹8,000-₹12,000 with "Include food" OFF
 2. Step 2: Shows 15 hostels (based on rent only)
 3. Step 3: Student toggles "Include food" ON
 4. Step 4: System recalculates: rent + average food cost per hostel
 5. Step 5: Some hostels now exceed ₹12,000 (total cost)
 6. Step 6: Result count updates: "10 hostels" (5 removed)
 7. Step 7: System shows: "5 hostels removed due to total cost exceeding range"
 8. Step 8: Student can adjust max price if needed

AC4: Quick Remove Filter

- Condition: Student wants to remove price filter after applying
1. Step 1: Price filter is active, showing filtered results
 2. Step 2: Active filter chip visible: "₹6k-₹12k X"
 3. Step 3: Student taps "X" on filter chip
 4. Step 4: System removes price filter immediately
 5. Step 5: Results refresh to show all hostels again (back to 47)
 6. Step 6: Filter chip disappears
 7. Step 7: Price slider resets to full range in filter panel

Exception Courses:

EX1: Backend Filter Service Timeout

- Condition: Backend takes too long to process filter request
1. Step 1: Student applies price filter
 2. Step 2: Backend doesn't respond within 5 seconds
 3. Step 3: System shows loading spinner with message: "Filtering results..."
 4. Step 4: If timeout exceeds 10 seconds:
 - System performs client-side filtering on already loaded results
 - Shows message: "Showing approximate results"
 5. Step 5: System logs backend timeout for investigation
 6. Step 6: Student sees filtered results (may be incomplete if not all results were loaded)

EX2: Price Data Missing for Some Hostels

- Condition: Some hostels have NULL or missing price data
- 1. Step 1: Student applies price filter ₹5,000-₹10,000
- 2. Step 2: Backend encounters hostels without price data
- 3. Step 3: System excludes hostels with missing prices from filtered results
- 4. Step 4: System logs: "3 hostels excluded due to missing price data"
- 5. Step 5: System shows disclaimer: "Some hostels excluded due to incomplete data"
- 6. Step 6: Student can still view complete unfiltered list if needed

EX3: Network Loss During Filter Application

- Condition: Internet disconnects while applying filter
- 1. Step 1: Student adjusts price slider and taps "Apply"
- 2. Step 2: Request sent to backend but network drops
- 3. Step 3: System detects network failure
- 4. Step 4: System falls back to client-side filtering on cached results
- 5. Step 5: Shows offline indicator: "Offline - showing cached results"
- 6. Step 6: Filter works but only on already-loaded hostels
- 7. Step 7: When connection restored, system re-applies filter from backend

EX4: Conflicting Filters Create Impossible Criteria

- Condition: Price filter conflicts with other active filters
- 1. Step 1: Student has already applied: "Luxury Amenities" + "Private Room" filters
- 2. Step 2: Student adds price filter: "Under ₹5,000"
- 3. Step 3: System detects conflict (luxury private rooms rarely under ₹5k)
- 4. Step 4: System shows: "0 hostels match all filters"
- 5. Step 5: System highlights conflicting filters
- 6. Step 6: System suggests: "Remove 'Luxury Amenities' to see 4 more options"
- 7. Step 7: Student can remove conflicting filters one-by-one

Postconditions:

Success:

- Only hostels within selected price range are displayed
- Active price filter chip visible with option to remove
- Result count updated to reflect filtered total
- Filter preference saved to student's profile for future searches
- Analytics logged: "price_filter_applied" with range values
- AI learns student's budget preference for recommendations
- Student can add more filters or proceed to view hostel details
- Filtered results maintain sort order (Recommended, Nearest, etc.)

Failure:

- Student sees error message with retry option
- Original unfiltered results remain visible
- Filter panel remains open for adjustment
- Error logged for debugging

USE CASE 4: Apply Advanced Filters (Amenities, Location, Preferences)

Element: Detail

Name: Apply Multiple Advanced Filters to Refine Hostel Search

ID: UC-STU-004

Description: The student wants to narrow down search results using multiple specific criteria such as amenities (WiFi, AC, food), location preferences (radius, transport connectivity), room configuration (sharing type), and personal preferences (culture, lifestyle). The system allows combining multiple filters simultaneously and shows real-time result counts as each filter is applied or removed.

Actors: Student (Primary), Filter Service (Secondary), Backend Server (Secondary), AI Recommendation Engine (Secondary)

Organization Benefits:

- Significantly improves match quality between students and hostels
- Reduces post-booking dissatisfaction and cancellations
- Increases conversion rate by showing only relevant options
- Provides valuable data on student preferences for market insights
- Enables personalized recommendations based on filter patterns
- Reduces support queries by helping students find exactly what they need

Preconditions:

- Student must have performed initial search showing results
- Filter panel/screen must be accessible
- At least 10+ hostels in initial results for meaningful filtering
- All hostel data (amenities, location, features) must be available in database
- Device must have internet connectivity
- Student may optionally have a saved filter preset

Triggers:

- Student taps "Advanced Filters" or "More Filters" button
- Student taps "Filters" and scrolls past basic options

- AI prompts: "Looking for specific amenities? Try advanced filters"
- Student taps "Refine Search" after viewing results
- System suggests: "Too many results? Add more filters"

Main Course:

1. Step 1: Student is viewing initial search results (e.g., 52 hostels)
2. Step 2: Student taps "Filters" button to open filter panel
3. Step 3: System displays comprehensive filter panel organized in expandable sections:
 - Location & Distance (collapsed/expanded)
 - Price & Payment (collapsed/expanded)
 - Hostel Type & Room (collapsed/expanded)
 - Amenities & Facilities (collapsed/expanded)
 - Food Services (collapsed/expanded)
 - Personal Preferences (collapsed/expanded)
 - Ratings & Reviews (collapsed/expanded)
 - Availability (collapsed/expanded)
4. Step 4: Student starts with Location & Distance section:
 - 4a. Taps to expand section
 - 4b. Sees radius slider (current: 5km, range: 1-20km)
 - 4c. Adjusts radius to 3km
 - 4d. Real-time count updates: "32 hostels within 3km"
 - 4e. Sees additional location options:
 - "Specific localities" (multi-select checkboxes)
 - Checks: "Kamla Nagar", "Mukherjee Nagar"
 - "Transport connectivity" checkboxes
 - Checks: "Metro within 1km"
 - 4f. Count updates: "18 hostels match"
5. Step 5: Student expands Amenities & Facilities:
 - 5a. Sees categorized amenity checkboxes:
 - Basic: WiFi, AC, Hot Water, Power Backup, Laundry
 - Study & Recreation: Study Room, Gym, Common Room
 - Security: CCTV, Security Guard, Biometric Entry
 - Parking: Two-wheeler, Four-wheeler
 - 5b. Student selects:
 - ✓ WiFi
 - ✓ AC
 - ✓ Study Room
 - ✓ CCTV
 - 5c. Each selection updates count dynamically:
 - After WiFi: "15 hostels"
 - After AC: "11 hostels"
 - After Study Room: "7 hostels"
 - After CCTV: "6 hostels"
6. Step 6: Student expands Hostel Type & Room:

- 6a. Sees room sharing options (radio buttons or checkboxes):
 - Single Room
 - Double Sharing
 - Triple Sharing
 - 4+ Sharing
- 6b. Student selects: "Double Sharing"
- 6c. Count updates: "4 hostels"
- 6d. Additional room preferences:
 - ✓ Attached bathroom
 - ✓ Balcony
- 7. Step 7: Student expands Food Services:
 - 7a. Options appear:
 - Food included (mandatory)
 - Food included (optional)
 - No food service
 - Meal type preferences: Veg only, Non-veg available, Jain food
 - 7b. Student selects: "Food included (mandatory)" + "Veg only"
 - 7c. Count updates: "3 hostels"
- 8. Step 8: Student expands Personal Preferences:
 - 8a. Sees lifestyle options:
 - Study-focused environment
 - Social/outgoing community
 - Quiet/peaceful atmosphere
 - Pet-friendly
 - LGBTQ+ friendly
 - 8b. Student selects: "Study-focused environment"
 - 8c. Count remains: "3 hostels" (already matched)
- 9. Step 9: Student expands Ratings & Reviews:
 - 9a. Minimum rating slider: 1★ to 5★
 - 9b. Student sets: 4★ minimum
 - 9c. Count updates: "2 hostels"
 - 9d. Additional option: "Verified reviews only" checkbox
- 10. Step 10: Student reviews all selected filters at top of panel:
 - Active filters summary:
 - "3km radius"
 - "Kamla Nagar, Mukherjee Nagar"
 - "Metro within 1km"
 - "WiFi, AC, Study Room, CCTV"
 - "Double Sharing, Attached bathroom"
 - "Food included (Veg)"
 - "Study-focused"
 - "4★+"
 - Result count: "2 hostels match all filters"
- 11. Step 11: Student taps "Apply Filters" or "Show Results" button

12. Step 12: Filter panel closes
13. Step 13: Search results screen updates:
 - Header: "2 hostels near Delhi University"
 - Active filter chips displayed (scrollable row):
 - "3km X"
 - "Kamla Nagar X"
 - "Metro <1km X"
 - "WiFi X"
 - "AC X"
 - "+4 more filters"
 - Only 2 matching hostels shown
14. Step 14: Student can tap any chip's "X" to remove that filter
15. Step 15: Student can tap "Clear All Filters" to reset
16. Step 16: Student can save current filter combination as preset: "My Ideal Hostel"
17. Step 17: Next time, student can load preset with one tap

Alternate Course:

AC1: Save Filter Preset

- Condition: Student wants to reuse complex filter combination
1. Step 1: Student has applied 8+ filters, found good results
 2. Step 2: Taps "Save Filters" button in filter panel
 3. Step 3: Dialog appears: "Name this filter preset"
 4. Step 4: Student enters: "Affordable Study Hostels"
 5. Step 5: Preset saved to student's profile
 6. Step 6: Next search, student can tap "Load Preset" → "Affordable Study Hostels"
 7. Step 7: All filters instantly applied

AC2: Progressive Filter Narrowing

- Condition: Student starts broad, narrows progressively
1. Step 1: Initial search: 52 hostels
 2. Step 2: Applies price filter: 32 hostels
 3. Step 3: Adds WiFi requirement: 28 hostels
 4. Step 4: Adds AC: 18 hostels
 5. Step 5: Adds Study Room: 7 hostels
 6. Step 6: Adds 4★ rating: 3 hostels
 7. Step 7: System shows progress: "You've narrowed from 52 to 3 hostels"
 8. Step 8: Student satisfied with 3, applies filters

AC3: Filter Suggestions Based on Availability

- Condition: System notices better results with slight adjustments
1. Step 1: Student's filters result in 1 hostel only

2. Step 2: System analyzes which filter is most restrictive
3. Step 3: System suggests: "Remove 'Balcony' requirement to see 5 more hostels"
4. Step 4: Student can tap suggestion to auto-adjust
5. Step 5: OR keep current strict filters

AC4: Batch Remove Conflicting Filters

- Condition: Multiple filters conflict
- 1. Step 1: Student has: "Under ₹5k" + "Private Room" + "Food included" + "AC"
- 2. Step 2: Results: 0 hostels
- 3. Step 3: System identifies conflicts:
 - Private AC rooms with food rarely under ₹5k
- 4. Step 4: System highlights conflicting filters in red
- 5. Step 5: Suggests: "Remove 2 filters to see results"
- 6. Step 6: Provides "Auto-adjust" button that removes least important filters based on AI
- 7. Step 7: Student can accept or manually adjust

AC5: Mobile vs Desktop Filter UI

- Condition: Different screen sizes need different UX
- Mobile:
 1. Full-screen filter panel
 2. One section visible at a time
 3. "Apply" button at bottom (sticky)
 4. Swipe down to close
- Desktop:
 1. Sidebar filter panel (doesn't cover results)
 2. Multiple sections visible, scroll within panel
 3. Filters apply in real-time (no "Apply" button needed)
 4. Results update as student adjusts filters

Exception Courses:

EX1: Too Many Filters Cause Performance Issues

- Condition: Student selects 15+ filters, backend slow
- 1. Step 1: Student adds 15th filter
- 2. Step 2: Backend query becomes complex, takes >5 seconds
- 3. Step 3: System shows: "Processing filters..." with spinner
- 4. Step 4: If exceeds 10 seconds, system suggests:
 - "Too many filters. Try removing some for faster results."
- 5. Step 5: System performs partial filtering client-side
- 6. Step 6: Logs performance issue for optimization

EX2: Filter Data Inconsistent

- Condition: Some hostels have incomplete amenity data
1. Step 1: Student filters for "WiFi" + "Study Room"
 2. Step 2: Backend finds 10 hostels with WiFi data, but only 5 have Study Room data populated
 3. Step 3: System shows 5 hostels that definitively match both
 4. Step 4: Adds note: "Some hostels excluded due to incomplete data"
 5. Step 5: System flags missing data for admin to complete

EX3: Saved Preset Becomes Invalid

- Condition: Student loads old preset with deprecated filters
1. Step 1: Student loads preset saved 6 months ago
 2. Step 2: Preset includes filter option that no longer exists
 3. Step 3: System removes invalid filter, applies rest
 4. Step 4: Shows message: "1 filter removed (no longer available)"
 5. Step 5: Student can update and re-save preset

EX4: Real-time Count Calculation Fails

- Condition: Dynamic count updates stop working
1. Step 1: Student adjusts filter, count should update
 2. Step 2: Count service times out or errors
 3. Step 3: System shows: "Calculating..." indefinitely
 4. Step 4: After 5 seconds, hides count display
 5. Step 5: Student can still apply filters (works without count preview)
 6. Step 6: Final results shown after "Apply"

Postconditions:

Success:

- Only hostels matching ALL selected filters are displayed
- Active filter chips visible with individual remove options
- Result count accurately reflects filtered total
- Filter combination can be saved as preset for future use
- Analytics logged with all filter selections
- AI learns student's detailed preferences
- Student has highly targeted, relevant results
- Decision-making significantly simplified

Failure:

- Student sees error message with suggestions
- Partial filtering may be applied
- Option to clear all filters and start over
- Error logged for system improvement

- Student can still view unfiltered results

USE CASE 5: AI-Powered Smart Recommendations

Element: Detail

Name: Get Personalized Hostel Recommendations Using AI

ID: UC-STU-005

Description: The system uses AI/machine learning algorithms to analyze the student's profile, search history, preferences, and behavior patterns to provide personalized hostel recommendations. The AI considers factors like budget compatibility, location preferences, lifestyle match with current residents, and predicted satisfaction probability to rank results intelligently.

Actors: Student (Primary), AI Recommendation Engine (Secondary), Backend Server (Secondary), User Profile Service (Secondary)

Organization Benefits:

- Dramatically increases conversion rate through better matches
- Reduces decision fatigue with curated recommendations
- Improves student satisfaction post-booking
- Reduces bounce rate by showing relevant options first
- Provides competitive advantage over basic search platforms
- Enables data-driven insights into student preferences
- Reduces customer support queries about "best fit"
- Increases student trust and platform loyalty

Preconditions:

- Student must have an account (logged in)
- Student profile should have basic information (budget, university, preferences)
- AI model must be trained and operational
- Student may have previous search/browsing history (optional but improves recommendations)
- Internet connectivity required
- Sufficient hostel data available for meaningful recommendations

Triggers:

- Student performs any search (location/university based)
- Student opens app home screen (shows personalized suggestions)
- Student taps "Recommended for You" section

- System automatically applies "Recommended" sort to search results
- Student has browsed 5+ hostels without booking (AI suggests better matches)

Main Course:

1. Step 1: Student logs into HMS Core app
2. Step 2: AI Recommendation Engine immediately loads student profile:
 - Profile data:
 - University: Delhi University
 - Budget range: ₹8,000-₹12,000
 - Room preference: Double sharing
 - Stated preferences: Study-focused, vegetarian food
 - Behavioral data (from history):
 - Previously searched: Kamla Nagar area (3 times)
 - Viewed hostels: Primarily with WiFi and AC
 - Time spent: Longer on hostels with study rooms
 - Saved favorites: 2 hostels (both near metro)
3. Step 3: Student performs search: "Hostels near Delhi University"
4. Step 4: Backend retrieves 47 matching hostels
5. Step 5: AI Recommendation Engine processes each hostel:
 - Calculates compatibility scores based on:
 - Budget match: How well price fits student's range (30% weight)
 - Location preference: Proximity to preferred areas (20% weight)
 - Amenity match: Availability of WiFi, AC, Study Room (25% weight)
 - Lifestyle compatibility: Study-focused environment score (15% weight)
 - Roommate compatibility: Match with current residents' profiles (10% weight)
6. Step 6: AI ranks hostels by total compatibility score (0-100%)
7. Step 7: System displays results with "Recommended" sort active by default
8. Step 8: Each hostel card shows:
 - Standard info: Name, price, distance, rating
 - NEW: Match percentage badge: "92% Match" (in green/gold)
 - NEW: Top match reason: "Great for studying, fits your budget"
9. Step 9: Top 3 results have special "Highly Recommended" badge
10. Step 10: Student sees personalized header:
 - "47 hostels near Delhi University - Sorted by best match for you"
11. Step 11: Student can tap match percentage to see detailed breakdown:
 - Modal/popup appears:
 - "Why is this a 92% match?"
 - Budget: ✓ "Within your ₹8k-₹12k range" (₹9,500)
 - Location: ✓ "In your preferred Kamla Nagar area" (850m)
 - Amenities: ✓ "Has WiFi, AC, Study Room you prefer"
 - Environment: ✓ "Study-focused atmosphere"
 - Residents: ~ "Good match with current students"
 - Close button or "Book Now" CTA

12. Step 12: Student can change sort if desired:
 - Dropdown: "Sort by: Recommended, Nearest, Price (Low-High), Rating"
 - Switching to "Nearest" removes match percentages, uses distance
13. Step 13: As student browses, AI learns:
 - Tracks which hostels student clicks on
 - Measures time spent on each hostel detail page
 - Notes which hostels are saved to favorites
 - Observes filter adjustments
14. Step 14: AI refines future recommendations based on observed behavior
15. Step 15: Student returns next day, sees updated recommendations based on yesterday's browsing

Alternate Course:

AC1: New User With Minimal Profile Data

- Condition: Student just signed up, no history available
1. Step 1: Student performs first search
 2. Step 2: AI has limited data (only university and basic profile)
 3. Step 3: AI uses general popularity-based ranking:
 - Popularity among students from same university (40% weight)
 - Overall ratings and reviews (30% weight)
 - Price appropriateness for area (30% weight)
 4. Step 4: Results shown without match percentages initially
 5. Step 5: Header: "Popular hostels near [University]"
 6. Step 6: As student browses, AI quickly learns preferences
 7. Step 7: After viewing 3-5 hostels, AI starts showing personalized suggestions

AC2: Student Actively Contradicts AI Recommendations

- Condition: Student consistently ignores top AI picks
1. Step 1: AI recommends budget-friendly study-focused hostels
 2. Step 2: Student repeatedly views luxury social hostels instead
 3. Step 3: AI detects pattern mismatch (stated vs revealed preferences)
 4. Step 4: AI adapts algorithm to weigh behavior over stated preferences
 5. Step 5: Next search, AI shows more luxury/social options
 6. Step 6: System may prompt: "We noticed you prefer [X]. Update your preferences?"
 7. Step 7: Student can update profile or continue

AC3: Student Requests Explanation of Ranking

- Condition: Student curious about AI logic
1. Step 1: Student taps "Why these recommendations?"
 2. Step 2: Explanatory screen appears:
 - "We recommend hostels based on:"

- Your budget: ₹8k-₹12k
 - Your university: Delhi University
 - Your preferences: Study-focused, Vegetarian
 - Your past searches: Kamla Nagar area
 - Similar students' choices: Students like you chose these
3. Step 3: Option to adjust preferences
 4. Step 4: "Recalculate" button to refresh recommendations

AC4: AI Suggests Alternative Options

- Condition: Student's ideal criteria too restrictive
1. Step 1: Student's perfect match criteria returns only 1 hostel
 2. Step 2: AI suggests "Similar hostels you might like" section
 3. Step 3: Shows 5 hostels that match 80-85% (vs 95% for top result)
 4. Step 4: Each shows: "89% match - slightly farther but better amenities"
 5. Step 5: Student can explore alternatives without changing filters

AC5: Group Recommendation for Friends

- Condition: Multiple students searching together
1. Step 1: Student invites 2 friends to joint search
 2. Step 2: Friends join via shared link
 3. Step 3: AI analyzes all 3 profiles
 4. Step 4: Finds hostels that match group consensus:
 - Budget range that works for all
 - Location acceptable to all
 - Amenities that majority prefer
 5. Step 5: Shows "Group Match" percentage for each hostel
 6. Step 6: Highlights: "Best for all 3 of you: 85% group match"

Exception Courses:

EX1: AI Service Temporarily Down

- Condition: AI recommendation engine unavailable
1. Step 1: Student performs search
 2. Step 2: AI service doesn't respond within 2 seconds
 3. Step 3: System falls back to rule-based sorting:
 - Sort by: distance (40%) + price match (30%) + rating (30%)
 4. Step 4: No match percentage badges shown
 5. Step 5: System shows: "Showing standard results (personalization temporarily unavailable)"
 6. Step 6: Engineers notified to fix AI service
 7. Step 7: Student experience degraded but not blocked

EX2: AI Produces Nonsensical Rankings

- Condition: Bug causes AI to rank obviously poor matches highly
1. Step 1: AI ranks a ₹25,000 hostel as top match for student with ₹8k budget
 2. Step 2: Backend validation layer catches extreme mismatch
 3. Step 3: System logs error: "AI score validation failed"
 4. Step 4: Falls back to rule-based sorting for that search
 5. Step 5: AI team receives alert to investigate model
 6. Step 6: Student sees normal results without bad recommendation

EX3: Roommate Compatibility Data Missing

- Condition: Hostel has residents but no personality data
1. Step 1: AI attempts roommate matching
 2. Step 2: Current residents haven't completed preference profiles
 3. Step 3: AI skips roommate compatibility scoring for this hostel
 4. Step 4: Uses other factors (location, budget, amenities) only
 5. Step 5: Match percentage may be lower than deserved
 6. Step 6: System notes: "Roommate compatibility unknown"

EX4: Student Profile Data Contradictory

- Condition: Student's behavior doesn't match stated preferences
1. Step 1: Profile says: budget ₹8k-₹10k, prefers budget-friendly
 2. Step 2: Behavior: consistently views ₹15k+ luxury hostels
 3. Step 3: AI detects contradiction between stated vs revealed preferences
 4. Step 4: AI weighs behavior more than profile (revealed preference theory)
 5. Step 5: Recommends mid-tier hostels (₹12k-₹14k) as compromise
 6. Step 6: System subtly suggests: "Update your budget preferences?"
 7. Step 7: Learns actual preferences over time

Postconditions:

Success:

- Student sees hostels ranked by personalized compatibility, not generic metrics
- Each hostel shows match percentage and explanation
- Top recommendations have "Highly Recommended" badges
- Detailed match breakdown available on hostel detail pages
- Student feels understood and trusts recommendations
- Decision-making time reduced significantly
- Higher likelihood of student satisfaction post-booking
- AI model improves from student's interaction data
- Student can provide feedback to refine recommendations
- Analytics logged with recommendation scores and student actions

Failure:

- System falls back to standard sorting (distance, price, rating)
- Error logged for AI team investigation
- Student still gets functional results
- No personalization badges shown
- System shows brief explanation of why personalization unavailable

USE CASE 6: Voice Search for Hostels

Element: Detail

Name: Search Hostels Using Voice Commands

ID: UC-STU-006

Description: The student uses voice input to search for hostels instead of typing. The system uses speech recognition to convert voice to text, natural language processing to understand the query intent and extract parameters (location, budget, preferences), and returns relevant hostel results. Supports multiple languages and conversational queries.

Actors: Student (Primary), Speech Recognition Service (Secondary), Natural Language Processing Engine (Secondary), Backend Search Service (Secondary)

Organization Benefits:

- Improves accessibility for students with disabilities
- Reduces friction in search process (voice faster than typing)
- Enables hands-free searching (while commuting, etc.)
- Appeals to younger, tech-savvy demographic
- Supports multilingual students (Hindi, regional languages)
- Differentiates from competitor apps
- Provides data on natural language query patterns

Preconditions:

- Student must have app installed and opened
- Device must have working microphone
- Student must grant microphone permission to app
- Internet connectivity required for speech processing
- Speech recognition service must be operational
- NLP model must be trained for hostel search domain

Triggers:

- Student taps microphone icon in search bar
- Student taps "Voice Search" button on home screen

- Student uses voice assistant integration: "Hey Google, find hostels on HMS Core"
- System suggests: "Try voice search for hands-free searching"

Main Course:

1. Step 1: Student opens HMS Core app on home screen
2. Step 2: Student sees microphone icon in/near main search bar
3. Step 3: Student taps microphone icon
4. Step 4: System requests microphone permission (if not granted previously)
5. Step 5: Student grants permission
6. Step 6: Voice input interface opens:
 - Large animated microphone icon (pulsing)
 - Text: "Listening..." or "Speak now"
 - Visual waveform showing sound input level
 - Language selector (English/Hindi/Regional) if not auto-detected
7. Step 7: Student speaks query naturally:
 - Example 1: "Show me boys hostels near IIT Delhi under 10 thousand rupees"
 - Example 2: "Delhi University ke paas saste hostel dikhao" (Hindi)
 - Example 3: "Find affordable hostels with WiFi and food near my college"
8. Step 8: Speech recognition service captures audio
9. Step 9: Audio sent to speech-to-text API (Google/AWS/Azure)
10. Step 10: API returns transcribed text:
 - "show me boys hostels near IIT Delhi under 10 thousand rupees"
11. Step 11: System displays transcription on screen:
 - "Did you say: 'show me boys hostels near IIT Delhi under 10 thousand rupees'?"
12. Step 12: Provides quick edit options:
 - "✓ Yes, search" button
 - "✗ Try again" button
 - Manual edit text field (if mis-heard)
13. Step 13: Student taps "✓ Yes, search" (or system auto-searches after 2 seconds if no action)
14. Step 14: Transcribed text sent to NLP Engine
15. Step 15: NLP Engine parses query to extract parameters:

```
{
  "location": "IIT Delhi",
  "location_type": "university",
  "gender": "boys",
  "max_price": 10000,
  "currency": "INR",
  "intent": "search_hostels"
}
```

16. Step 16: NLP handles variations and synonyms:

- "cheap" / "affordable" / "saste" → budget-friendly filter
 - "near" / "close to" / "ke paas" → location proximity
 - "ten thousand" / "10k" / "10000" → 10,000
 - "IIT Delhi" / "IIT" → matches university in database
17. Step 17: Extracted parameters sent to backend search service
18. Step 18: Backend executes search with parameters:
- Location: IIT Delhi coordinates
 - Gender filter: Boys hostels
 - Price filter: Max ₹10,000
19. Step 19: Backend returns matching hostels (e.g., 23 results)
20. Step 20: Voice search interface closes
21. Step 21: Results screen displays:
- Header: "23 boys hostels near IIT Delhi under ₹10,000"
 - Shows applied filters automatically
 - Lists matching hostels
22. Step 22: Student can browse results normally or refine with voice again
23. Step 23: System saves voice query to search history for learning

Alternate Course:

AC1: Ambiguous or Incomplete Voice Query

- Condition: Student's voice query lacks some parameters
1. Step 1: Student says: "Show me cheap hostels"
 2. Step 2: NLP detects missing location
 3. Step 3: System responds with voice + text:
 - Voice: "Where are you looking for hostels?"
 - Text: Displays "Where?" with location suggestions
 4. Step 4: Student can speak location or tap suggestion
 5. Step 5: Student says: "Near Delhi University"
 6. Step 6: System confirms: "Searching for affordable hostels near Delhi University"
 7. Step 7: Search executes with complete parameters

AC2: Incorrect Speech Recognition

- Condition: System misunderstands spoken query
1. Step 1: Student says: "IIT Delhi"
 2. Step 2: System transcribes as: "IT deli"
 3. Step 3: Displays: "Did you say 'IT deli'?"
 4. Step 4: Student taps "X Try again"
 5. Step 5: Microphone re-activates
 6. Step 6: Student speaks more clearly or uses manual edit
 7. Step 7: Student can type "IIT Delhi" in edit field
 8. Step 8: Taps "Search" and query executes correctly

AC3: Multilingual Voice Search (Hindi Example)

- Condition: Student prefers speaking in Hindi
- 1. Step 1: Student taps language selector, chooses "Hindi"
- 2. Step 2: System activates Hindi speech recognition
- 3. Step 3: Student speaks: "Delhi University ke paas sasta ladkiyon ka hostel dikhao jisme khana mile"
- 4. Step 4: Speech-to-text returns: "delhi university ke paas sasta ladkiyon ka hostel dikhao jisme khana mile"
- 5. Step 5: NLP (trained on Hindi) extracts:

```
{  
  "location": "Delhi University",  
  "gender": "girls",  
  "price_preference": "budget",  
  "amenities": ["food_included"]  
}
```

- 6. Step 6: Search executes with Hindi parameters
- 7. Step 7: Results displayed (UI can be Hindi or English based on preference)

AC4: Complex Multi-Criteria Voice Query

- Condition: Student specifies many parameters in one query
- 1. Step 1: Student says: "Find me a girls hostel near Jamia University with WiFi AC and food within 12000 rupees with good reviews"
- 2. Step 2: NLP extracts all parameters:
 - Location: Jamia University
 - Gender: Girls
 - Amenities: WiFi, AC, Food included
 - Max price: ₹12,000
 - Rating: High (interpreted as 4+ stars)
- 3. Step 3: System confirms: "Searching for highly-rated girls hostels near Jamia University with WiFi, AC, and food under ₹12,000"
- 4. Step 4: Executes complex filtered search
- 5. Step 5: All filters automatically applied in results

Exception Courses:

EX1: Microphone Permission Denied

- Condition: Student denies microphone access
- 1. Step 1: Student taps voice search icon
- 2. Step 2: System requests permission, student taps "Deny"
- 3. Step 3: Voice interface doesn't open

4. Step 4: System shows message: "Microphone access required for voice search"
5. Step 5: Provides "Settings" button to enable permission
6. Step 6: Falls back to text search
7. Step 7: Voice search unavailable until permission granted

EX2: No Speech Detected

- Condition: Student doesn't speak or speaks too quietly
1. Step 1: Voice interface activated, listening...
 2. Step 2: No audio input detected for 5 seconds
 3. Step 3: System prompts: "I didn't hear anything. Please speak clearly."
 4. Step 4: Continues listening for 5 more seconds
 5. Step 5: If still no input, closes voice interface
 6. Step 6: Student can retry or use text search

EX3: Noisy Environment Interferes

- Condition: Background noise makes speech unclear
1. Step 1: Student speaks but loud traffic/music in background
 2. Step 2: Speech recognition returns garbled/nonsense text
 3. Step 3: NLP cannot extract meaningful parameters
 4. Step 4: System detects poor confidence score (<60%)
 5. Step 5: Shows: "I couldn't understand that clearly. Background noise might be interfering."
 6. Step 6: Suggests: "Try again in a quieter place or use text search"
 7. Step 7: Provides "Try Again" button

EX4: Speech Recognition Service Down

- Condition: Speech-to-text API unavailable
1. Step 1: Student speaks query
 2. Step 2: API request times out or returns error
 3. Step 3: System detects service failure
 4. Step 4: Shows: "Voice search temporarily unavailable"
 5. Step 5: Automatically converts to text search
 6. Step 6: Opens keyboard for manual input
 7. Step 7: Error logged for technical team

EX5: NLP Cannot Parse Query

- Condition: Student's query too vague or nonsensical
1. Step 1: Student says: "Find me something good"
 2. Step 2: Speech recognized correctly
 3. Step 3: NLP cannot extract meaningful parameters
 4. Step 4: System responds: "I need more information. Where are you looking?"
 5. Step 5: Engages in conversational clarification

6. Step 6: Student provides more details
7. Step 7: Once sufficient info gathered, executes search

Postconditions:

Success:

- Student's voice query accurately converted to search parameters
- Relevant hostel results displayed matching voice intent
- Applied filters visible (auto-generated from voice)
- Voice query saved to search history
- Faster search experience than typing
- System learns voice patterns for improved future recognition
- Analytics logged: "voice_search_completed" with language and query
- Student can refine results or search again

Failure:

- Student directed to text search alternative
- Clear error message shown
- Retry option available
- No data lost if voice fails
- Error logged for service improvement

USE CASE 7: View Hostel on Map & Explore Nearby

Element: Detail

Name: View Search Results on Interactive Map

ID: UC-STU-007

Description: The student switches from list view to map view to visualize hostel locations geographically. The system displays hostels as pins/markers on an interactive map with clustering, color-coding by availability/price, and ability to explore nearby facilities like markets, metro stations, and the university campus. Student can tap markers for quick info and adjust search area by dragging the map.

Actors: Student (Primary), Map Service (Secondary - Google Maps/OpenStreetMap), Backend Server (Secondary), Geocoding Service (Secondary)

Organization Benefits:

- Helps students understand geographic context better than list
- Reduces questions about "how far is X from Y"

- Increases engagement through interactive exploration
- Reduces post-booking surprises about location
- Enables discovery of hostels student might miss in list view
- Differentiates from basic directory apps
- Provides heatmap data on popular search areas

Preconditions:

- Student must have performed a search with results
- All hostels in results must have valid geocoordinates
- Map service API must be accessible
- Internet connectivity required for map tiles
- Device GPS optional but helpful for "current location" feature

Triggers:

- Student taps "Map View" toggle/button on search results screen
- Student taps map icon on individual hostel card
- System defaults to map view if student previously preferred it
- Student wants to see distance relationships visually

Main Course:

1. Step 1: Student is viewing search results in list view (e.g., 28 hostels near Delhi University)
2. Step 2: Student taps "Map" toggle button (top-right, switches from "List")
3. Step 3: System initiates map view loading:
 - Shows loading indicator
 - Prepares geocoordinate data for all 28 hostels
4. Step 4: Map view interface appears:
 - Map Canvas:
 - Centered on search location (Delhi University)
 - Zoom level auto-set to show all results
 - University Marker:
 - Special pin/icon for Delhi University (blue star or building icon)
 - Labeled: "Delhi University"
 - Hostel Markers:
 - 28 pins representing hostels
 - Color-coded based on status:
 - Green: Available & within budget
 - Yellow: Available but above budget
 - Orange: Limited beds (1-2 left)
 - Red: Fully occupied/not available
 - Blue: Highly recommended by AI
 - Marker Clustering (if many hostels close together):

- If 5+ hostels within small radius, shows single cluster marker
 - Cluster displays number: "12" (hostels in cluster)
 - Tapping/zooming in breaks cluster into individual pins
5. Step 5: Price labels appear on/near markers:
 - Each hostel pin shows price: "₹9,500"
 - Helps quick price comparison without opening details
 6. Step 6: Map controls available:
 - Zoom In/Out buttons (+ / -)
 - Current Location button (GPS icon) - centers map on student
 - Layers button:
 - Toggle traffic layer
 - Toggle transit layer (metro/bus lines)
 - Toggle satellite view
 - Filter button (same filters as list view)
 - List View toggle (to switch back)
 7. Step 7: Student taps a hostel marker
 8. Step 8: Mini info card appears (bottom sheet or popup):
 - "Green Valley Boys Hostel"
 - Thumbnail image
 - Price: ₹9,500/month
 - Distance: 1.2 km from Delhi University
 - Rating: 4.3★ (67 reviews)
 - Available beds: 3
 - Quick amenity icons: WiFi, Food, AC
 - Buttons:
 - "View Details" (opens full hostel page)
 - "Directions" (opens Google Maps navigation)
 - "Call" (contact owner)
 - Heart icon (save to favorites)
 9. Step 9: Student can swipe card left/right to see adjacent hostel cards without closing
 10. Step 10: Student explores map by dragging:
 - Pans to see nearby areas
 - Moves beyond original search radius
 11. Step 11: "Search this area" button appears (floating at top):
 - Shows: "Search this area (14 hostels)"
 - Count updates as student drags map
 12. Step 12: Student taps "Search this area"
 13. Step 13: System:
 - Captures new map center coordinates
 - Captures new visible map bounds
 - Queries backend for hostels in new area
 - Updates markers on map
 14. Step 14: New hostels appear as markers, old ones remain or fade
 15. Step 15: Student can also explore nearby facilities:

- Taps "Nearby" button in map controls
 - Selects category:
 - Metro stations
 - Bus stops
 - Markets/shopping
 - Hospitals
 - Restaurants
 - Coaching centers
16. Step 16: Selected facility markers appear on map with different icons
- Example: Student selects "Metro stations"
 - 3 metro station icons appear with labels
 - Student can see which hostels are near metro
17. Step 17: Student can draw/measure distance:
- Taps "Measure Distance" tool
 - Taps point A (a hostel), then point B (metro station)
 - Line drawn showing distance: "850 meters"
18. Step 18: Student switches back to list view when ready:
- Taps "List" toggle
 - List view shows hostels in order of distance from current map center

Alternate Course:

AC1: Marker Clustering Interaction

- Condition: Many hostels in small area create cluster
1. Step 1: Map shows cluster marker with "8"
 2. Step 2: Student taps cluster marker
 3. Step 3: Two options:
 - Option A: Cluster "explodes" into 8 individual markers (if zoom allows)
 - Option B: Map auto-zooms in until cluster breaks apart
 4. Step 4: Student can now see individual hostel markers
 5. Step 5: Tapping individual marker shows info card as normal

AC2: Filter While in Map View

- Condition: Student wants to apply filters without switching to list
1. Step 1: Student viewing map with 28 hostel markers
 2. Step 2: Student taps "Filters" button on map
 3. Step 3: Filter panel slides in (same as list view)
 4. Step 4: Student adjusts price: ₹8k-₹10k, adds WiFi filter
 5. Step 5: Student taps "Apply"
 6. Step 6: Map markers update instantly:
 - Hostels not matching filter fade out or disappear
 - Only 12 matching hostels remain visible
 7. Step 7: Filter chips displayed above map

8. Step 8: Student continues exploring with filtered results

AC3: Compare Multiple Hostels on Map

- Condition: Student wants to compare distance/location of shortlisted hostels
1. Step 1: Student has saved 3 hostels to favorites
 2. Step 2: Opens map view
 3. Step 3: Taps "Show Favorites" option
 4. Step 4: Only favorited hostels appear as markers (3 pins)
 5. Step 5: Different pin color/shape for favorites (gold star)
 6. Step 6: Student can see relative distances visually
 7. Step 7: Can measure distances between favorites and university
 8. Step 8: Helps make final decision based on location

AC4: Get Directions to Hostel

- Condition: Student wants navigation to visit a hostel
1. Step 1: Student taps hostel marker on map
 2. Step 2: Mini card appears
 3. Step 3: Student taps "Directions" button
 4. Step 4: System options:
 - Option A: Opens in Google Maps app (if installed) with navigation
 - Option B: Shows directions within HMS app (if navigation feature built)
 5. Step 5: Student sees route from current location to hostel
 6. Step 6: Shows estimated time: "12 minutes by auto" / "25 minutes walking"
 7. Step 7: Student can start turn-by-turn navigation

AC5: Save Current Map View

- Condition: Student wants to return to same map view later
1. Step 1: Student has explored map, found good area
 2. Step 2: Taps "Save View" or bookmark button
 3. Step 3: System saves:
 - Map center coordinates
 - Zoom level
 - Active filters
 4. Step 4: Saved as "Mukherjee Nagar area" in student's profile
 5. Step 5: Next time, student can load saved view instantly

Exception Courses:

EX1: Map Service API Failure

- Condition: Google Maps / map tiles unavailable
1. Step 1: Student taps "Map View"
 2. Step 2: Map tiles fail to load (network error / API down)

3. Step 3: System shows blank/gray map with error message
4. Step 4: Message: "Map temporarily unavailable. Showing list view instead."
5. Step 5: Automatically falls back to list view
6. Step 6: Student can still browse hostels, just without map
7. Step 7: "Try Map Again" button available

EX2: Missing Geocoordinates for Some Hostels

- Condition: Some hostels lack lat/long data
1. Step 1: Of 28 hostels, 3 have null coordinates
 2. Step 2: System plots 25 hostels on map
 3. Step 3: Shows disclaimer: "3 hostels not shown (location data incomplete)"
 4. Step 4: Excluded hostels still appear in list view
 5. Step 5: System flags missing data for admin to fix

EX3: GPS Permission Denied for Current Location

- Condition: Student taps "Current Location" but hasn't granted permission
1. Step 1: Student taps GPS icon to center on their location
 2. Step 2: System requests location permission
 3. Step 3: Student denies
 4. Step 4: Map stays centered on search location (university)
 5. Step 5: "Current Location" feature disabled
 6. Step 6: Student can still use all other map features

EX4: Slow Network Causes Lag

- Condition: Map tiles load slowly due to poor connection
1. Step 1: Student switches to map view
 2. Step 2: Map tiles load progressively (low resolution first)
 3. Step 3: Hostel markers appear before map fully loads
 4. Step 4: Student sees pins on gray/loading tiles
 5. Step 5: System shows loading indicator
 6. Step 6: High-res tiles load within 10-15 seconds
 7. Step 7: If timeout, falls back to lower resolution

EX5: Too Many Markers Overwhelm Map

- Condition: 100+ hostels in search results
1. Step 1: Student searches broad area, gets 150 hostels
 2. Step 2: System detects excessive markers
 3. Step 3: Enforces aggressive clustering
 4. Step 4: Shows max 20-30 visible markers/clusters at current zoom
 5. Step 5: As student zooms in, more detail revealed
 6. Step 6: Suggests: "Apply filters to narrow results"
 7. Step 7: System remains performant and usable

Postconditions:**Success:**

- Student sees geographic distribution of hostels clearly
- Understands relative distances from university and other landmarks
- Can explore nearby facilities (metro, markets, etc.)
- Quick access to hostel details via marker taps
- Can switch between map and list views seamlessly
- Map state (center, zoom) remembered if student switches away
- Student makes more location-informed decision
- Analytics logged: "map_view_used", interaction patterns tracked

Failure:

- System gracefully falls back to list view
- Student can still complete search/booking process
- Error logged for technical resolution
- Clear messaging about why map unavailable

USE CASE 8: Save Hostels to Favorites/Wishlist

Element: Detail

Name: Save Hostels to Favorites for Later Review

ID: UC-STU-008

Description: The student can save interesting hostels to a favorites/wishlist for easy access later. This allows comparison, sharing with parents, and revisiting without re-searching. The system allows organizing favorites into custom lists, adding personal notes, and receiving notifications about saved hostels (price drops, availability changes).

Actors: Student (Primary), Backend Server (Secondary), Notification Service (Secondary)

Organization Benefits:

- Increases user engagement and return visits
- Extends decision-making time without losing interest
- Provides data on which hostels students seriously consider
- Enables remarketing to students who favorited but didn't book
- Helps students organize research systematically
- Increases conversion by keeping options accessible

Preconditions:

- Student must be logged into account
- Student must have viewed at least one hostel
- Backend must support favorites storage
- Internet connectivity required to save favorites
- Favorites data synced across devices

Triggers:

- Student taps heart/bookmark icon on hostel card
- Student taps "Save" button on hostel detail page
- Student wants to remember a hostel for later
- System prompts: "Save this hostel to compare later?"

Main Course:

1. Step 1: Student is browsing search results or viewing hostel details
2. Step 2: Student finds a hostel of interest: "Sunshine Boys Hostel"
3. Step 3: Student sees heart icon (outline/empty) on hostel card
4. Step 4: Student taps heart icon
5. Step 5: System immediately saves hostel to favorites:
 - Heart icon fills/changes color (outline → solid red/gold)
 - Brief animation (heart beat or scale up)
 - Haptic feedback (subtle vibration on mobile)
 - Toast notification: "Added to Favorites" with undo option
6. Step 6: Backend API call:

POST /api/favorites/add

```
{
  "student_id": "12345",
  "hostel_id": "H9876",
  "timestamp": "2026-01-15T14:30:00Z"
}
```

7. Step 7: Hostel saved to student's favorites list in database
8. Step 8: Student continues browsing and saves 2 more hostels similarly
9. Step 9: Student wants to review saved hostels
10. Step 10: Student navigates to "Favorites" section:
 - Taps heart icon in main navigation/tab bar
 - Or accesses from profile menu: "My Favorites"
11. Step 11: Favorites screen opens showing:
 - Header: "My Favorites (3)"
 - Options:
 - "Compare" button (if 2+ favorites)
 - "Create List" button

- Sort dropdown: Recently added, Price low-high, Distance, Rating
- Hostel Cards: (same format as search results)
 - Sunshine Boys Hostel - ₹9,500/month, 1.2km
 - Green Valley PG - ₹8,800/month, 2.1km
 - Scholar's Den - ₹11,200/month, 0.8km
- Each card shows:
 - Filled heart icon (can tap to unfavorite)
 - "Add Note" button
 - "Move to List" dropdown
 - Saved date: "Saved 2 days ago"
- 12. Step 12: Student taps "Add Note" on "Sunshine Boys Hostel"
- 13. Step 13: Note input dialog appears:
 - Text field: "Add personal notes..."
 - Character limit: 500
- 14. Step 14: Student types: "Parents liked this one. Close to metro. Visit on Saturday."
- 15. Step 15: Student taps "Save Note"
- 16. Step 16: Note saved and displayed below hostel card as snippet
- 17. Step 17: Student wants to organize favorites better
- 18. Step 18: Student taps "Create List" button
- 19. Step 19: Dialog appears: "Name your list"
- 20. Step 20: Student types: "Top Choices"
- 21. Step 21: Student taps "Create"
- 22. Step 22: New list created, student can move hostels into it
- 23. Step 23: Student selects 2 hostels (checkboxes) and taps "Move to 'Top Choices'"
- 24. Step 24: Hostels moved to custom list, can be accessed from lists dropdown
- 25. Step 25: Student can switch between "All Favorites" and specific lists like "Top Choices", "Backup Options"

Alternate Course:

AC1: Remove from Favorites

- Condition: Student changes mind about saved hostel
- 1. Step 1: Student viewing favorites list
- 2. Step 2: Taps filled heart icon on a hostel card
- 3. Step 3: Heart icon empties/changes back to outline
- 4. Step 4: Toast: "Removed from Favorites" with undo option (5 seconds)
- 5. Step 5: Hostel removed from list (slides out animation)
- 6. Step 6: If student taps "Undo" within 5 seconds, hostel restored
- 7. Step 7: Backend deletes favorite entry if undo not used

AC2: Quick Compare from Favorites

- Condition: Student wants to compare 2-3 saved hostels
- 1. Step 1: Student in Favorites screen with 3+ hostels

2. Step 2: Taps "Compare" button
3. Step 3: Checkboxes appear on each hostel card
4. Step 4: Student selects 2 hostels to compare
5. Step 5: "Compare (2)" button activates at bottom
6. Step 6: Student taps "Compare (2)"
7. Step 7: Opens side-by-side comparison view (same as UC-Comparison)
8. Step 8: Student can see all differences: price, amenities, distance, etc.

AC3: Share Favorites with Parents

- Condition: Student wants parent input on shortlisted hostels
1. Step 1: Student viewing "Top Choices" list (2 hostels)
 2. Step 2: Taps "Share List" button
 3. Step 3: Share options appear:
 - WhatsApp, Email, SMS, Copy Link
 4. Step 4: Student selects WhatsApp
 5. Step 5: System generates shareable link with list data
 6. Step 6: WhatsApp opens with pre-filled message:
 - "Check out these hostels I'm considering: [Link]"
 7. Step 7: Student sends to parents
 8. Step 8: Parents can view hostels (read-only) without app/account
 9. Step 9: Parents can add comments via shared interface

AC4: Price Drop Notification

- Condition: Saved hostel reduces price
1. Step 1: Student saved "Green Valley PG" at ₹8,800/month
 2. Step 2: Two days later, owner reduces price to ₹8,200
 3. Step 3: System detects price change in nightly batch job
 4. Step 4: System sends push notification:
 - "💰 Price dropped! Green Valley PG now ₹8,200/month (was ₹8,800)"
 5. Step 5: Student taps notification
 6. Step 6: Opens hostel detail page showing new price
 7. Step 7: Banner: "Price decreased by ₹600 since you saved this"
 8. Step 8: Student can book at new lower price

AC5: Availability Alert

- Condition: Full hostel becomes available
1. Step 1: Student saved "Scholar's Den" which had 0 beds available
 2. Step 2: A student cancels, 1 bed now available
 3. Step 3: System sends notification: "🛏️ Bed available at Scholar's Den!"
 4. Step 4: Student can act quickly before others book
 5. Step 5: First-come-first-serve from favorites alerts

Exception Courses:

EX1: Network Failure While Saving

- Condition: Internet drops when student taps heart
 1. Step 1: Student taps heart icon to favorite hostel
 2. Step 2: API call fails (no network)
 3. Step 3: Heart icon appears filled (optimistic UI)
 4. Step 4: System queues save operation locally
 5. Step 5: Shows offline indicator: "Will sync when online"
 6. Step 6: When connection restored:
 - Auto-syncs favorite to backend
 - Shows: "Favorites synced"
 7. Step 7: If sync fails repeatedly, reverts heart to empty with error

EX2: Maximum Favorites Limit Reached

- Condition: Student tries to save 100+ hostels (system limit)
 1. Step 1: Student has 100 favorites (system max to prevent abuse)
 2. Step 2: Student tries to favorite another hostel
 3. Step 3: System shows message: "Favorites limit reached (100)"
 4. Step 4: Suggests: "Remove some favorites or organize into lists"
 5. Step 5: Provides link to Favorites management
 6. Step 6: Student must remove at least one before adding new

EX3: Hostel Deleted/Delisted After Saving

- Condition: Saved hostel no longer available on platform
 1. Step 1: Student opens Favorites after 1 week
 2. Step 2: One saved hostel has been delisted by owner
 3. Step 3: System detects hostel_id invalid
 4. Step 4: Shows hostel card grayed out with message:
 - "This hostel is no longer available"
 5. Step 5: Provides "Remove" button
 6. Step 6: Student can remove from favorites
 7. Step 7: System suggests similar alternatives

EX4: Sync Conflict Across Devices

- Condition: Student favorites on phone and web simultaneously
 1. Step 1: Student saves Hostel A on phone
 2. Step 2: Before sync, student saves Hostel B on web
 3. Step 3: Both devices try to sync favorites
 4. Step 4: Backend detects potential conflict
 5. Step 5: Uses merge strategy: combines both favorites
 6. Step 6: Both devices end up with Hostel A + B favorited

7. Step 7: No data loss, favorites merged correctly

Postconditions:

Success:

- Hostel saved to student's favorites/wishlist
- Heart icon reflects saved state across app
- Favorite accessible from Favorites section anytime
- Student can add notes, organize into lists
- Can compare favorites side-by-side
- Receives notifications about price/availability changes
- Data synced across all devices
- Analytics logged: "hostel_favorited" with hostel_id
- Student can share favorites with others

Failure:

- Error message shown with retry option
- Local queue created for offline sync
- Student can still view hostel details
- Error logged for debugging

HMS Core - Additional Student Use Cases (Detailed Documentation)

USE CASE 9: View Complete Hostel Details

Element: Detail

Name: View Comprehensive Hostel Information

ID: UC-STU-009

Description: The student views detailed information about a specific hostel including high-resolution images, complete amenities list, current resident reviews, location context, room availability, pricing breakdown, hostel rules, and AI-generated compatibility insights. The page provides all information needed to make an informed decision.

Actors: Student (Primary), Backend Server (Secondary), Image Service (Secondary), Review System (Secondary), AI Recommendation Engine (Secondary)

Organization Benefits:

- Reduces pre-booking queries to owners/admin
- Builds confidence through transparency
- Decreases bounce rate with comprehensive info
- Reduces post-booking cancellations due to unmet expectations
- Provides data on which information students prioritize
- Enables informed decision-making leading to higher satisfaction

Preconditions:

- Student must have searched and found hostels
- Hostel must exist in database with complete information
- Images must be uploaded and accessible
- Reviews must be available (if any)
- Internet connectivity required
- Hostel must be active/published

Triggers:

- Student taps hostel card from search results
- Student taps hostel from favorites

- Student taps hostel from comparison view
- Student follows shared hostel link
- Student taps "View Details" on map marker

Main Course:

1. Step 1: Student taps on "Sunshine Boys Hostel" card from search results
2. Step 2: Loading transition animation (card expands to full screen)
3. Step 3: Hostel detail page loads progressively:
 - First (Immediate - Above Fold):
 - Hero image gallery appears
 - Navigation elements: Back arrow, Share icon, Heart icon
 - Image counter: "1 / 15"
 - Loading State:
 - Skeleton screens for content below images
 - Shimmer effect on placeholder cards
4. Step 4: Image Gallery Section loads (top of page):
 - Features:
 - Full-width swipeable image carousel
 - 15 high-resolution images:
 - Exterior shots (building facade, entrance)
 - Room photos (different room types)
 - Common areas (study room, dining hall, lounge)
 - Facilities (bathroom, kitchen, terrace)
 - Amenities (gym, parking, security setup)
 - Pinch to zoom functionality
 - Dots indicator at bottom
 - Tap to enter fullscreen gallery mode
 - Image captions: "Double Sharing Room", "Study Area"
5. Step 5: Basic Information Section appears:

SUNSHINE BOYS HOSTEL

[Verified ✓] [Boys] [PG]

₹9,500 / month

+ ₹2,000 security deposit (refundable)

📍 1.2 km from Delhi University

★ 4.3 (127 reviews)

🛏️ 3 beds available

[90% AI Match - Excellent] (green badge)

6. Step 6: Quick Info Cards Section (Grid layout):

- Card 1: Room Type
 - Icon: Bed
 - "Double Sharing"
 - "2 beds per room"
- Card 2: Bathroom
 - Icon: Shower
 - "Attached Bathroom"
 - "Hot water 24/7"
- Card 3: Food
 - Icon: Utensils
 - "3 Meals Included"
 - "Pure Vegetarian"
- Card 4: Availability
 - Icon: Calendar
 - "Available Now"
 - "Join anytime"

7. Step 7: AI Match Breakdown Section (Expandable):

- Header: "Why we recommend this (90% match)" [Expand ▼]
- Student taps to expand:

Perfect for you because:

✓ Budget Match (100%)

₹9,500 fits your ₹8k-₹12k range perfectly

✓ Location Convenience (95%)

Only 1.2km from Delhi University

10-minute bike ride or 15-minute walk

Metro station 800m away

✓ Roommate Compatibility (85%)

2 current residents highly compatible:

- Raj (21, CS 3rd year) - Same course!

Similar sleep schedule (11pm - 7am)

Also from Punjab

- Amit (22, Physics 4th year)

Quiet, studious personality match

High cleanliness rating

✓ Lifestyle Match (90%)

Quiet study environment you prefer

Dedicated study room available

No late-night parties (hostel rule)

✓ Amenities Match (88%)

Has all your must-haves:

WiFi (50 Mbps) ✓

AC in rooms ✓

Study room ✓

CCTV security ✓

Veg food ✓

⚠ Consider:

- No gym (you marked as nice-to-have)
- Common bathroom on 2nd floor (your room on 3rd)
- 10:30pm curfew on weekdays

8. Step 8: Pricing Breakdown Section (Expandable):

- Header: "Monthly Cost Breakdown" [Expand ▼]

Base Rent: ₹6,500

Food (3 meals): ₹2,500

Electricity (actual): ₹300

WiFi: ₹200

Total per month: ₹9,500

One-time costs:

Security Deposit: ₹2,000 (refundable)

Registration Fee: ₹500

First month total: ₹12,000

9. Step 9: Amenities & Facilities Section:

- Organized in Categories:
- Basic Amenities:
 - ✓ High-speed WiFi (50 Mbps)
 - ✓ Air Conditioning
 - ✓ Hot Water (24/7)
 - ✓ Power Backup
 - ✓ Washing Machine
 - ✓ Iron & Ironing Board
- Food Services:
 - ✓ Breakfast (7:30-9:30am)
 - ✓ Lunch (1:00-3:00pm)
 - ✓ Dinner (7:30-9:30pm)
 - ✓ Pure Vegetarian
 - ✓ Dining Hall
 - ✗ Kitchen Access
- Study & Recreation:
 - ✓ Study Room (6am-11pm)
 - ✓ Common Lounge
 - ✓ TV Room
 - ✓ Indoor Games (Carrom, Chess)
 - ✗ Gym

- ✓ Terrace Access
- Security:
 - ✓ 24/7 CCTV (12 cameras)
 - ✓ Security Guard
 - ✓ Biometric Entry
 - ✓ Visitor Register
 - ✓ Fire Extinguishers
 - ✓ Emergency Exit
- Parking:
 - ✓ Two-wheeler (covered)
 - ✗ Four-wheeler
 - ✓ Bicycle Stand

10. Step 10: Room Details Section:

- Available Room Types:
- [Tab: Double Sharing] [Tab: Triple Sharing]
- Double Sharing (Selected):
 - Room size: 150 sq ft
 - 2 single beds
 - 2 study tables with chairs
 - 2 wardrobes
 - Attached bathroom
 - 1 window
 - AC unit
 - 3 rooms available
 - Price: ₹9,500/month
- [View Room Photos] button

11. Step 11: About This Hostel Section:

Description:

Sunshine Boys Hostel is a premium accommodation facility located in the heart of North Campus area. Established in 2019, we cater specifically to university students with a focus on creating a conducive study environment while maintaining a friendly community atmosphere.

[Read More ▼]

Hostel Rules:

- Curfew: 10:30pm weekdays, 11:30pm weekends
- Guests allowed: 6pm-8pm only (register at desk)
- Smoking/alcohol strictly prohibited
- Quiet hours: 10pm-7am
- Monthly room inspection for cleanliness
- 1-month notice required for vacating

Timings:

- Office: 9am-8pm daily
- Meals: Breakfast 7:30-9:30am, Lunch 1-3pm, Dinner 7:30-9:30pm
- Study Room: 6am-11pm

12. Step 12: Location Section:

- Interactive map showing hostel location
- Distance markers to key places:
 - Delhi University: 1.2 km
 - GTB Nagar Metro: 800m
 - Main Market: 500m
- [Get Directions] button

13. Step 13: Reviews Section:

- Overall Rating: 4.3★ (127 reviews)
- Rating Breakdown:
 - Cleanliness: 4.5★
 - Food Quality: 4.0★
 - Safety: 5.0★
 - Value for Money: 4.2★

- WiFi Speed: 4.4★
 - Recent Reviews (sorted by date):
 - Review cards with student name, rating, date, comment
 - [Read All Reviews] button
- Step 14: Current Residents Section (Privacy-Protected):
 - Shows anonymized profiles of compatible roommates
 - "2 students similar to you currently living here"
 - Option to "Request to Chat" with residents
 - Step 15: Similar Hostels Section:
 - "Students also viewed these hostels"
 - 3-4 hostel cards with quick compare option
 - Step 16: Bottom Action Bar (Sticky):
 - [❤ Save] [Share] [Compare] [Book Now]
 - Step 17: Student scrolls through all sections, reviews information
 - Step 18: Student taps "Book Now" to proceed with booking

Alternate Course:

AC1: View in Fullscreen Gallery Mode

- Condition: Student wants to examine photos closely
- Step 1: Student taps any image in gallery
 - Step 2: Enters fullscreen immersive mode
 - Step 3: Can swipe through all 15 images
 - Step 4: Pinch to zoom on details
 - Step 5: Image captions shown at bottom
 - Step 6: Tap X or back to exit fullscreen

AC2: Contact Current Residents

- Condition: Student wants to ask residents about experience
- Step 1: Student taps "Chat with Residents"
 - Step 2: System sends connection request to residents
 - Step 3: If accepted, opens group chat or individual chats

4. Step 4: Student can ask questions directly
5. Step 5: Residents respond at their convenience

AC3: Get Directions for Visit

- Condition: Student wants to physically visit hostel
1. Step 1: Student taps "Get Directions" button
 2. Step 2: Opens Google Maps with route
 3. Step 3: Shows walking/driving/transit options
 4. Step 4: Student can start navigation
 5. Step 5: Estimated time displayed: "15 min by auto"

AC4: Share Hostel with Parents

- Condition: Student wants parent approval before booking
1. Step 1: Student taps Share icon
 2. Step 2: Share options appear: WhatsApp, Email, SMS, Copy Link
 3. Step 3: Student selects WhatsApp
 4. Step 4: Pre-filled message with hostel link
 5. Step 5: Parents receive link, can view all details
 6. Step 6: Parents can comment or approve

AC5: View Food Menu/Schedule

- Condition: Student curious about exact meal offerings
1. Step 1: In Food Services section, taps "View Menu"
 2. Step 2: Weekly menu sheet opens:
 - Monday: Breakfast (Paratha, Chai), Lunch (Dal, Rice, Roti), Dinner (Paneer, Roti)
 - Tuesday: [Similar format]
 - etc.
 3. Step 3: Can see nutritional info if available
 4. Step 4: Student closes menu

Exception Courses:

EX1: Slow Image Loading

- Condition: Poor network causes images to load slowly
1. Step 1: Gallery shows placeholder/blur
 2. Step 2: Progressive loading: low-res first, then high-res
 3. Step 3: Loading indicator on images still loading
 4. Step 4: Text content loads independently (doesn't wait for images)
 5. Step 5: Student can read details while images load

EX2: Hostel No Longer Available

- Condition: Hostel was delisted after student opened link
1. Step 1: Student opens hostel detail page
 2. Step 2: System detects hostel inactive
 3. Step 3: Shows banner: "This hostel is no longer accepting bookings"
 4. Step 4: Displays reason if available: "All rooms filled" or "Temporarily closed"
 5. Step 5: Suggests similar alternatives
 6. Step 6: "View Similar Hostels" button

EX3: Reviews Section Empty

- Condition: New hostel with zero reviews
1. Step 1: Student scrolls to Reviews section
 2. Step 2: Shows: "No reviews yet. Be the first!"
 3. Step 3: Explains: "This is a newly listed hostel"
 4. Step 4: Shows other trust indicators:
 - Verified badge
 - Years in business
 - Current occupancy rate
 5. Step 5: Student can proceed cautiously or look elsewhere

EX4: AI Match Data Unavailable

- Condition: Student hasn't completed preference profile

1. Step 1: AI Match section shows generic message
2. Step 2: "Complete your profile for personalized insights"
3. Step 3: "Quick Preferences" button (2-minute quiz)
4. Step 4: If student completes, page refreshes with AI data
5. Step 5: Match percentage and reasons now displayed

EX5: Owner Contact Information Missing

- Condition: Phone number not available for direct call
1. Step 1: Student taps "Call" button
 2. Step 2: System detects no phone number
 3. Step 3: Shows: "Contact via in-app chat instead"
 4. Step 4: Opens chat interface with owner
 5. Step 5: Student can send inquiry message

Postconditions:

Success:

- Student has comprehensive understanding of hostel
- All questions answered through detailed information
- Student can make informed decision
- Time spent on page tracked (engagement metric)
- Student proceeds to: save, compare, share, or book
- Analytics logged: "detail_page_viewed", time_spent, sections_viewed
- Trust built through transparency and complete info

Failure:

- Student shown error message with retry option
 - Partial data displayed if available
 - Student directed back to search results
 - Error logged for technical team
-

USE CASE 10: Compare Multiple Hostels Side-by-Side

Element: Detail

Name: Compare Up to 3 Hostels Simultaneously

ID: UC-STU-010

Description: The student selects 2-3 hostels from search results or favorites to view side-by-side comparison of all key parameters including price, location, amenities, reviews, and AI compatibility scores. The system highlights differences, shows best value, and provides smart insights to aid decision-making.

Actors: Student (Primary), Backend Server (Secondary), AI Comparison Engine (Secondary)

Organization Benefits:

- Reduces decision paralysis through clear comparison
- Increases conversion by facilitating final choice
- Reduces bounce rate from indecision
- Provides data on which factors drive decisions
- Helps students feel confident in their choice
- Reduces post-booking regret

Preconditions:

- Student must have found at least 2 hostels of interest
- Hostels must have complete data for comparison
- Internet connectivity required
- Student can compare from: search results, favorites, or detail pages

Triggers:

- Student checks "Compare" boxes on 2+ hostel cards
- Student taps "Compare" button with 2+ hostels selected
- Student taps "Compare" from favorites screen
- Student adds hostel to comparison from detail page

Main Course:

1. Step 1: Student browsing search results, has narrowed down to 3 hostels
2. Step 2: Student taps "Compare" checkbox on first hostel card:

- "Sunshine Boys Hostel"
- Checkbox fills with checkmark
- Floating button appears at bottom: "Compare (1/3)"

3. Step 3: Student selects second hostel:

- "Green Valley PG"
- Button updates: "Compare (2/3)"
- Button becomes active/clickable (needs min 2)

4. Step 4: Student selects third hostel:

- "Scholar's Den"
- Button updates: "Compare (3/3)" - maximum reached
- Other compare checkboxes become disabled

5. Step 5: Student taps "Compare (3/3)" button

6. Step 6: Comparison view loads with transition animation

7. Step 7: Comparison screen displays (table/card format):

- Mobile View: Swipeable horizontal cards
- Desktop View: Side-by-side table

8. Step 8: Header Section:

Comparing 3 Hostels

[Remove] [Add Another] [Share Comparison]

9. Step 9: Hostel Cards Header (Top Row):

SUNSHINE	GREEN	SCHOLAR'S
BOYS	VALLEY PG	DEN
[Image]	[Image]	[Image]
4.3★ (127)	4.1★ (89)	4.5★ (156)
[Remove ×]	[Remove ×]	[Remove ×]

10. Step 10: Price Comparison Section:

MONTHLY COST

₹9,500	₹8,800	₹11,200	
[LOWEST]	✓		

SECURITY DEPOSIT

₹2,000	₹1,500	₹3,000	
[LOWEST]	✓		

TOTAL FIRST MONTH

₹12,000	₹10,800	₹14,700	
[LOWEST]	✓		

11. Step 11: Location Comparison:

DISTANCE FROM UNIVERSITY

1.2 km	2.1 km	0.8 km	
	[NEAREST]	✓	
10 min	18 min	7 min	
bike	bike	bike	

METRO STATION

800m	1.2km	600m	
GTB Nagar	Vishwa	GTB Nagar	
	Vidyalaya	[NEAREST]	✓

12. Step 12: Room & Amenities Comparison:

ROOM TYPE

Double	Triple	Double	
Sharing	Sharing	Sharing	

BATHROOM

Attached	Common	Attached	
✓		✓	

AIR CONDITIONING

✓	X	✓	
---	---	---	--

WiFi SPEED

50 Mbps	20 Mbps	100 Mbps	
		[FASTEST]✓	

STUDY ROOM

✓	X	✓	
---	---	---	--

GYM

X	X	✓	
---	---	---	--

13. Step 13: Food Services Comparison:

MEALS INCLUDED

3 meals	2 meals	3 meals	
(B,L,D)	(B,D)	(B,L,D)	

FOOD TYPE

Veg Only	Veg + Non	Veg Only	
	Veg		

FOOD RATING

4.0★	3.8★	4.5★	
	[HIGHEST]✓		

14. Step 14: Ratings & Reviews Comparison:

OVERALL RATING

4.3★	4.1★	4.5★	
	[HIGHEST]✓		

NUMBER OF REVIEWS

127	89	156	
reviews	reviews	reviews	
	[MOST]✓		

SAFETY RATING

5.0★	4.2★	4.8★	
[HIGHEST]✓			

15. Step 15: AI Compatibility Comparison:

AI MATCH SCORE

90%	75%	88%	
Excellent	Good	Excellent	
[BEST] ✓			

ROOMMATE COMPATIBILITY

85%	60%	80%	
2 matches	1 match	2 matches	
[BEST] ✓			

16. Step 16: Availability Comparison:

AVAILABLE BEDS

3 beds	1 bed	5 beds	
	[LIMITED] Δ		

JOIN FROM

Anytime	Feb 1	Anytime	
---------	-------	---------	--

17. Step 17: AI Smart Insights Section (Bottom):

💡 SMART INSIGHTS

Best Value: GREEN VALLEY PG

- ₹700/month cheaper than Sunshine
- Saves ₹8,400 annually
- Trade-off: No AC, farther from university

Best Location: SCHOLAR'S DEN

- Closest to university (0.8km)
- Saves 10 min commute daily = 5 hrs/month
- Premium price (+₹1,700/month) for convenience

Best Overall Match: SUNSHINE BOYS HOSTEL

- Highest AI compatibility (90%)
- Best roommate matches for your personality
- Balanced price-amenities-location
- Perfect safety rating (5.0★)

💰 Cost Analysis:

Green Valley saves ₹700/month BUT:

- No AC (summer heat issue)
 - Extra commute time (~8 min/day)
 - Only breakfast+dinner (lunch separate cost ~₹2k)
- Real savings: ~₹500/month after lunch cost

📍 Location Analysis:

Scholar's Den is 400m closer BUT:

- ₹1,700/month premium
 - Saves ~5-7 min/trip
- Paying ₹243/hr for saved commute time
- Consider if time is critical

18. Step 18: Bottom Action Buttons:

[View	[View	[View			
Details]	Details]	Details]			
[Book Now]	[Book Now]	[Book Now]			

[Share Comparison] [Save Comparison] [Print]

19. Step 19: Student reviews all comparisons, reads smart insights
20. Step 20: Student decides "Sunshine Boys Hostel" best overall despite not being cheapest
21. Step 21: Student taps "Book Now" under Sunshine Boys Hostel
22. Step 22: Proceeds to booking flow

Alternate Course:

AC1: Remove Hostel from Comparison

- Condition: Student wants to replace one hostel with another
1. Step 1: Student taps "Remove ×" on Green Valley PG
 2. Step 2: That column disappears/slides out
 3. Step 3: Now comparing only 2 hostels
 4. Step 4: "Add Another" button becomes active
 5. Step 5: Student can select different hostel
 6. Step 6: Comparison refreshes with new hostel

AC2: Share Comparison with Parents

- Condition: Student wants parent input on final decision
1. Step 1: Student taps "Share Comparison" button
 2. Step 2: System generates comparison page link
 3. Step 3: Share sheet opens: WhatsApp, Email, SMS
 4. Step 4: Student sends to parents
 5. Step 5: Parents view interactive comparison
 6. Step 6: Parents can comment/suggest via link

AC3: Save Comparison for Later

- Condition: Student not ready to decide, wants to revisit
1. Step 1: Student taps "Save Comparison"
 2. Step 2: Comparison saved to profile
 3. Step 3: Named: "Comparison saved Jan 15"
 4. Step 4: Accessible from profile/favorites section
 5. Step 5: Student can return anytime to review

AC4: Filter Comparison View

- Condition: Student wants to see only specific parameters
1. Step 1: Student taps "Customize View" button
 2. Step 2: Checkboxes appear for categories:
 - ✓ Price
 - ✓ Location
 - ✗ Reviews (hide)
 - ✓ Amenities
 - ✗ AI Match (hide)
 3. Step 3: Comparison table updates to show only selected
 4. Step 4: Cleaner, focused view

AC5: Print/Export Comparison

- Condition: Student wants physical copy for parents
1. Step 1: Student taps "Print" button
 2. Step 2: Generates printer-friendly PDF
 3. Step 3: Can print or save as PDF
 4. Step 4: Formatted table with all data
 5. Step 5: Student can email PDF to parents

Exception Courses:

EX1: Only 1 Hostel Selected

- Condition: Student tries to compare with less than 2
1. Step 1: Student selects 1 hostel, taps Compare
 2. Step 2: Message: "Select at least 2 hostels to compare"
 3. Step 3: Compare button remains disabled
 4. Step 4: Student must select 1 more

EX2: Comparison Data Incomplete

- Condition: One hostel missing some data fields
1. Step 1: Comparison loads with some "N/A" cells
 2. Step 2: Missing data shown as "Info unavailable"
 3. Step 3: Disclaimer: "Some data incomplete"
 4. Step 4: Student can still compare available data
 5. Step 5: Can contact hostel for missing info

EX3: Network Failure During Comparison

- Condition: Internet drops while loading comparison
1. Step 1: Comparison page partially loads
 2. Step 2: Shows cached basic data (price, location)
 3. Step 3: Detailed data may be missing
 4. Step 4: Offline banner: "Limited data - reconnect for full comparison"
 5. Step 5: Retry button available

EX4: Hostels Too Similar

- Condition: 3 hostels very similar in all aspects
1. Step 1: Comparison shows minimal differences
 2. Step 2: AI detects similarity
 3. Step 3: Message: "These hostels are very similar"
 4. Step 4: Highlights the few differences that exist
 5. Step 5: Suggests: "Consider comparing with more varied options"

Postconditions:

Success:

- Student has clear side-by-side view of all key differences
- Decision-making significantly aided by visual comparison
- Smart insights help understand trade-offs
- Student feels confident in final choice
- Can easily share comparison with others
- Analytics logged: "comparison_viewed", hostels_compared, time_spent
- Student proceeds to book preferred hostel

Failure:

- Error message with retry option
- Student returns to search results or favorites
- Can still view individual hostel details
- Error logged for debugging

USE CASE 11: Book Hostel & Select Room

Element: Detail

Name: Complete Hostel Booking and Room Selection

ID: UC-STU-011

Description: The student initiates booking process for a chosen hostel, selects specific room and bed if available, provides required documents, chooses join date, reviews terms, makes payment (or schedules payment), and receives booking confirmation. The system guides through each step with validation and clear instructions.

Actors: Student (Primary), Hostel Owner/Warden (Secondary), Payment Gateway (Secondary), Backend Server (Secondary), Notification Service (Secondary)

Organization Benefits:

- Generates revenue through successful bookings
- Reduces booking abandonment with guided flow
- Captures necessary student data for owner/warden
- Processes secure payments
- Creates documented agreement between parties
- Reduces manual coordination effort
- Provides booking analytics and trends

Preconditions:

- Student must be logged in
- Student must have selected a hostel
- Hostel must have available beds
- Student profile must be reasonably complete
- Payment gateway must be operational
- Internet connectivity required

Triggers:

- Student taps "Book Now" button on hostel detail page
- Student taps "Book Now" from comparison view
- Student ready to commit after research

Main Course:

1. Step 1: Student on "Sunshine Boys Hostel" detail page, taps "Book Now"
2. Step 2: Booking flow begins with transition animation
3. Step 3: Step 1 of 6: Room & Bed Selection

SELECT YOUR ROOM

Available Room Types:

- Double Sharing - ₹9,500/month
 - └ 3 rooms available
 - └ Room 301, Room 305, Room 308
- Triple Sharing - ₹7,500/month
 - └ 1 room available
 - └ Room 204

4. Step 4: Student selects "Double Sharing"
5. Step 5: Room selection expands to show specific rooms:

ROOM 301 (Floor 3, South Wing)

[Room Photo]

- Window with street view
- Attached bathroom
- AC
- 2 beds (1 available)
- Current occupant: Raj K. (21, CS 3rd year)
- Available: Bed A (near window)

[Select This Room]

ROOM 305 (Floor 3, North Wing)

[Room Photo]

- Garden view
- Attached bathroom
- AC
- 2 beds (1 available)
- Current occupant: Amit S. (22, Physics 4th)
- Available: Bed B (near door)

[Select This Room]

ROOM 308 (Floor 3, East Wing)

[Room Photo]

- Quiet, faces back
- Attached bathroom
- AC
- 2 beds (2 available - empty room)
- No current occupant
- Choose: Bed A or Bed B

[Select This Room]

6. Step 6: Student selects Room 301 (good roommate compatibility with Raj)

7. Step 7: Bed assignment confirmed:

- "You've selected: Room 301, Bed A"
- [Continue to Next Step]

8. Step 8: Step 2 of 6: Join Date Selection

WHEN WOULD YOU LIKE TO JOIN?

[Calendar Picker]

Today: Jan 15, 2026

Quick Options:

- Immediately (Within 3 days)
- This Month (Jan 2026)
- Next Month (Feb 2026)
- Custom Date [Select Date]

Minimum Stay: 3 months

Notice Period: 1 month

9. Step 9: Student selects "This Month" and picks Jan 20, 2026

10. Step 10: Step 3 of 6: Personal Information

CONFIRM YOUR DETAILS

Full Name: [Auto-filled from profile]

Arjun Singh

Date of Birth: [DD/MM/YYYY]

15/03/2003

Phone Number: [Auto-filled]

+91 98765 43210

Emergency Contact:

Name: Rajesh Singh (Father)

Phone: +91 98765 12345

University/College:

Delhi University

Course & Year:

B.Tech Computer Science, 2nd Year

Permanent Address:

[Text Area]

11. Step 11: Student reviews and edits if needed, clicks Continue

12. Step 12: Step 4 of 6: Document Upload

UPLOAD REQUIRED DOCUMENTS

1. Photo ID Proof (Required) ⚠️

Accepted: Aadhaar, Passport, Driving License

[📎 Upload File] or [📷 Take Photo]

Status: ✗ Not Uploaded

2. Student ID Card (Required) ⚠️

University/College ID

[📎 Upload File] or [📷 Take Photo]

Status: ✗ Not Uploaded

3. Passport Size Photo (Required) ⚠️

Recent photograph

[📎 Upload File] or [📷 Take Photo]

Status: ✗ Not Uploaded

4. Parent ID Proof (Optional)

Guardian's Aadhaar/Passport

[📎 Upload File]

Status: - Not Required

Note: Documents will be verified by hostel owner

13. Step 13: Student uploads all required documents:

- Taps Upload, selects Aadhaar from gallery
- Status updates: ✓ Uploaded (2.3 MB)
- Repeats for student ID and photo

14. Step 14: All required documents uploaded, Continue button activates

15. Step 15: Step 5 of 6: Review & Terms

BOOKING SUMMARY

Hostel: Sunshine Boys Hostel

Room: 301 (Double Sharing, Floor 3)

Bed: Bed A (near window)

Join Date: Jan 20, 2026

PAYMENT BREAKDOWN

Monthly Rent: ₹9,500

Security Deposit: ₹2,000 (refundable)

Registration Fee: ₹500 (one-time)

Platform Fee: ₹200 (one-time)

Total Due Now: ₹12,200

Prorated for Jan 20-31: ₹3,467

+ Deposit + Fees: ₹2,700

Adjusted Total: ₹6,167

Next Payment (Feb 1): ₹9,500

TERMS & CONDITIONS

☐ I agree to hostel rules and regulations

[\[View Rules\]](#)

☐ I agree to 3-month minimum stay

☐ I understand 1-month notice required for
vacating (deposit will be refunded after
room inspection)

☐ I agree to HMS Core Terms of Service

[\[View Terms\]](#)

☐ I authorize HMS Core to share my details
with hostel owner for verification

16. Step 16: Student reads terms, checks all boxes

17. Step 17: Student taps "Proceed to Payment"

18. Step 18: Step 6 of 6: Payment

PAYMENT METHOD

Total Amount: ₹6,167

Select Payment Method:

○ UPI (Google Pay, PhonePe, Paytm)

[UPI ID: yourname@paytm]

○ Debit/Credit Card

[Card Details Form]

○ Net Banking

[Select Bank Dropdown]

○ Pay at Hostel (Cash/Card)

Pay directly to owner on join date

(Booking confirmed after owner approval)

 Secure Payment powered by Razorpay

19. Step 19: Student selects "UPI"

20. Step 20: Enters UPI ID: arjun@paytm

21. Step 21: Taps "Pay ₹6,167"

22. Step 22: Payment gateway modal opens

23. Step 23: Student approves payment in UPI app

24. Step 24: Payment processing... (loading spinner)

25. Step 25: Payment Successful! 

 **BOOKING CONFIRMED!**

Booking ID: #BK2026015-001

Transaction ID: TXN987654321

Your hostel booking is confirmed!

Sunshine Boys Hostel

Room 301, Bed A

Join Date: Jan 20, 2026

Amount Paid: ₹6,167

WHAT'S NEXT?

1. You'll receive confirmation email & SMS
2. Hostel owner will verify your documents
(within 24 hours)
3. You'll receive joining instructions via app
4. Visit hostel on Jan 20 with original documents

Owner Contact:

Mr. Sharma: +91 98765 00000

[\[View Booking Details\]](#)

[\[Download Receipt\]](#)

[\[Chat with Owner\]](#)

[\[Back to Home\]](#)

26. Step 26: Confirmation notifications sent:

- Push notification to student
- Email with booking details and receipt
- SMS with booking ID and owner contact
- Notification to hostel owner/warden
- Notification to student's linked parent account

27. Step 27: Student taps "View Booking Details"

28. Step 28: Booking details page opens with complete information and timeline

Alternate Course:

AC1: Pay at Hostel Option

- Condition: Student prefers to pay in person
1. Step 1: At payment step, student selects "Pay at Hostel"
 2. Step 2: No online payment required
 3. Step 3: Booking status: "Pending Owner Approval"
 4. Step 4: Owner receives booking request
 5. Step 5: Owner approves after reviewing profile
 6. Step 6: Student receives confirmation
 7. Step 7: Payment due on join date

AC2: Room Not Available Anymore

- Condition: Someone else books room during student's booking process
1. Step 1: Student completes room selection
 2. Step 2: Proceeds through steps
 3. Step 3: At payment, system checks availability
 4. Step 4: Room 301 Bed A now occupied
 5. Step 5: Alert: "Selected room no longer available"
 6. Step 6: System shows alternative rooms
 7. Step 7: Student can select different room or cancel

AC3: Partial Document Upload

- Condition: Student can't upload all documents immediately
1. Step 1: Student uploads 2 of 3 required docs
 2. Step 2: Tries to continue
 3. Step 3: Warning: "1 required document missing"
 4. Step 4: Option: "Upload Later" button
 5. Step 5: Booking proceeds with pending status
 6. Step 6: Must upload within 24 hours
 7. Step 7: Reminder notifications sent

AC4: Apply Promo Code/Discount

- Condition: Student has discount code

1. Step 1: At payment summary, sees "Have a promo code?"
2. Step 2: Taps to expand
3. Step 3: Enters code: "FIRST500"
4. Step 4: System validates code
5. Step 5: Discount applied: ₹500 off
6. Step 6: Total updates: ₹6,167 → ₹5,667
7. Step 7: Proceeds with discounted price

AC5: Split Payment (EMI Option)

- Condition: Large amount, student wants installments
1. Step 1: At payment step, sees "Pay in installments"
 2. Step 2: Options:
 - 50% now, 50% in 15 days
 - 3 monthly installments
 3. Step 3: Student selects option
 4. Step 4: Pays first installment
 5. Step 5: Booking confirmed with payment plan
 6. Step 6: Reminders sent for remaining payments

Exception Courses:

EX1: Payment Failure

- Condition: UPI transaction fails or times out
1. Step 1: Student approves payment in UPI app
 2. Step 2: Transaction fails (insufficient balance / network issue)
 3. Step 3: Error message: "Payment failed. Please try again"
 4. Step 4: Booking held for 15 minutes
 5. Step 5: Student can retry with same/different method
 6. Step 6: If retry succeeds, booking confirmed
 7. Step 7: If timeout, booking released

EX2: Document Upload Failure

- Condition: File size too large or format not supported
1. Step 1: Student selects 10MB PDF file
 2. Step 2: Upload fails: "File too large (max 5MB)"
 3. Step 3: Suggestions: "Compress image or use JPEG format"
 4. Step 4: Student can retry with smaller file
 5. Step 5: Or use camera to take new photo

EX3: Network Loss During Booking

- Condition: Internet disconnects mid-flow
1. Step 1: Student filling personal information
 2. Step 2: Network drops
 3. Step 3: Auto-save kicks in, data stored locally
 4. Step 4: Offline banner appears
 5. Step 5: When connection restored, data syncs
 6. Step 6: Student can continue from where left off

EX4: Owner Rejects Booking

- Condition: Owner reviews and declines (rarely happens)
1. Step 1: Booking pending owner approval
 2. Step 2: Owner rejects due to policy mismatch
 3. Step 3: Student notified: "Booking not approved"
 4. Step 4: Reason shown if provided by owner
 5. Step 5: Full refund processed automatically
 6. Step 6: Student can book different hostel

EX5: Gateway/Server Error

- Condition: Payment gateway down or backend error
1. Step 1: Student attempts payment

2. Step 2: Gateway doesn't respond
3. Step 3: Error: "Payment service temporarily unavailable"
4. Step 4: Booking details saved
5. Step 5: "Try again in a few minutes" message
6. Step 6: Student can retry or choose "Pay at Hostel"

Postconditions:

Success:

- Booking confirmed and saved in database
- Payment processed and recorded
- Room/bed allocated and marked unavailable for others
- Confirmation sent to student (app + email + SMS)
- Notification sent to hostel owner/warden
- Parent (if linked) notified of booking
- Documents uploaded for owner verification
- Booking appears in student's "My Bookings" section
- Analytics logged: "booking_completed", revenue tracked
- Student can access booking details anytime

Failure:

- Payment refunded if deducted
- Room/bed released back to available pool
- Student can retry booking
- Error details logged for investigation
- Support team notified if critical failure

USE CASE 12: File Complaint/Issue

Element: Detail

Name: Report Hostel Issues or Complaints

ID: UC-STU-012

Description: The student reports problems, issues, or complaints about their hostel stay (maintenance, cleanliness, food quality, roommate conflicts, safety concerns, etc.). The system categorizes the complaint using AI, routes to appropriate authority (warden/owner/admin), tracks resolution status, and provides timeline transparency.

Actors: Student (Primary), AI Categorization Engine (Secondary), Hostel Warden (Secondary), Hostel Owner (Secondary), HMS Core Admin (Secondary - escalation), Notification Service (Secondary)

Organization Benefits:

- Identifies and resolves issues quickly before escalation
- Provides data on common hostel problems for quality improvement
- Creates accountability for hostel management
- Builds trust through transparent complaint handling
- Reduces student churn by addressing concerns
- Provides legal documentation if needed
- Enables platform to monitor hostel quality

Preconditions:

- Student must be logged in
- Student must have active booking/stay at hostel
- Hostel warden/owner contact details must be available
- Internet connectivity required
- Complaint categories must be defined in system

Triggers:

- Student taps "File Complaint" or "Report Issue" in app
- Student faces problem requiring management attention
- Student accesses complaint section from booking details
- System prompts: "Having issues? Let us know"

Main Course:

1. Step 1: Student experiencing AC malfunction in Room 301
2. Step 2: Opens HMS Core app, navigates to "My Bookings"
3. Step 3: Taps on active booking: "Sunshine Boys Hostel"
4. Step 4: Booking detail page shows options, student taps "File Complaint"
5. Step 5: Complaint form opens:

FILE A COMPLAINT

What's the issue?

Category (Select one):

- ☐ Maintenance (AC, plumbing, electrical, etc.)
- ☐ Cleanliness (room, bathroom, common areas)
- ☐ Food Quality/Service
- ☐ Noise/Disturbance
- ☐ Safety/Security Concern
- ☐ Roommate Issue
- ☐ Hostel Staff Behavior
- ☐ Billing/Payment Dispute
- ☐ Other

[Next]

6. Step 6: Student selects "Maintenance"
7. Step 7: Sub-category appears:

Maintenance Issue - Select Type:

- ☐ Air Conditioning
- ☐ Plumbing (water, drainage)
- ☐ Electrical (lights, power)
- ☐ Furniture (bed, table, chair)
- ☐ Door/Window/Locks
- ☐ Other Maintenance

[Next]

8. Step 8: Student selects "Air Conditioning"

9. Step 9: Detailed complaint form:

DESCRIBE THE ISSUE

Issue: Air Conditioning

Title (Brief Summary):

[Text field]

Example: "AC not cooling in Room 301"

Description (Detailed):

[Text area - 500 char limit]

Please describe the issue in detail...

When did this start?

- ☐ Just now
- ☐ Today
- ☐ Yesterday
- ☐ 2-3 days ago
- ☐ More than 3 days ago

Priority:

- ☐  Urgent (Immediate attention needed)
- ☐  Normal (Can wait 24-48 hours)
- ☐  Low (Not urgent)

Attach Photos/Videos (Optional):

[ Upload Files] [ Take Photo]

Max 5 files, 10MB each

[Submit Complaint]

10. Step 10: Student fills form:

- Title: "AC not working since yesterday"
- Description: "AC in Room 301 stopped cooling since yesterday evening. Remote is working but unit not responding. Room getting very hot."
- Started: "Yesterday"
- Priority: "  Normal"
- Uploads 1 photo of AC unit

11. Step 11: Student taps "Submit Complaint"

12. Step 12: AI Categorization Engine processes:

- Category: Maintenance
- Sub-category: Air Conditioning
- Priority: Normal
- Location: Room 301, Sunshine Boys Hostel
- Suggested Route: Hostel Warden (Mr. Gupta)
- Urgency Score: Medium

13. Step 13: Complaint submitted successfully:

✅ COMPLAINT SUBMITTED

Complaint ID: #CMP2026015-042

Your complaint has been submitted and forwarded to:

Hostel Warden: Mr. Gupta

Expected Response: Within 24 hours

Expected Resolution: Within 48 hours

COMPLAINT DETAILS

Category: Maintenance - Air Conditioning

Title: AC not working since yesterday

Priority: Normal

Status: Submitted 🕒

TIMELINE

Jan 15, 11:30 AM - Complaint submitted

Awaiting warden acknowledgment...

[\[Track Status\]](#) [\[Add Update\]](#) [\[Chat with Warden\]](#)

[\[Back to Bookings\]](#)

14. Step 14: Notifications sent:

- To Student: "Complaint submitted successfully"
- To Warden (Mr. Gupta): Push + SMS + Email
 - "New complaint from Room 301: AC not working"
 - [Link to complaint details](#)
- To Hostel Owner: Email notification (copy)

15. Step 15: Warden receives notification, opens complaint
16. Step 16: Warden acknowledges complaint (taps "Acknowledge")
17. Step 17: Student receives update:
 - Push notification: "Your complaint has been acknowledged by warden"
 - Timeline updated:
 - Jan 15, 11:35 AM - Warden acknowledged
 - Status: In Progress 
18. Step 18: Warden assigns technician, adds comment: "Technician will visit by 3 PM today"
19. Step 19: Student sees update in complaint tracking:
 - Timeline:
 - Jan 15, 11:30 AM - Submitted
 - Jan 15, 11:35 AM - Acknowledged
 - Jan 15, 11:40 AM - Warden comment: "Technician visiting at 3 PM"
 - Status: In Progress 
20. Step 20: Technician fixes AC
21. Step 21: Warden marks complaint as "Resolved"
 - Adds resolution note: "AC gas refilled. Working now."
22. Step 22: Student receives notification:
 - "Your complaint has been marked as resolved"
23. Step 23: Student opens complaint, sees:

COMPLAINT RESOLVED 

Complaint ID: #CMP2026015-042

Resolution: AC gas refilled. Working now.

Resolved by: Warden (Mr. Gupta)

Resolved on: Jan 15, 3:45 PM

Total Time: 4 hours 15 minutes

FEEDBACK

Was this issue resolved satisfactorily?

[ Yes, Satisfied] [ No, Not Satisfied]

Additional Comments (Optional):

[Text field]

[Submit Feedback] [Reopen Complaint]

24. Step 24: Student taps "  Yes, Satisfied"
25. Step 25: Complaint closed with positive feedback
26. Step 26: Analytics logged, warden performance tracked

Alternate Course:

AC1: Urgent Priority Complaint

- Condition: Student selects urgent priority
1. Step 1: Student selects "  Urgent" priority
 2. Step 2: System flags for immediate attention
 3. Step 3: Instant notification to Warden AND Owner
 4. Step 4: Phone call option appears: "Call Warden Now"
 5. Step 5: Expected response: Within 2 hours
 6. Step 6: Auto-escalates if no acknowledgment in 2 hours

AC2: Anonymous Complaint

- Condition: Student fears retaliation for sensitive complaint

1. Step 1: At submission, sees checkbox: "☐ Submit anonymously"
2. Step 2: Student checks anonymous option
3. Step 3: Warning: "Your identity won't be shared, but this may delay resolution"
4. Step 4: Complaint submitted without student details to warden
5. Step 5: Only complaint ID and room number visible to warden

AC3: Complaint Against Warden/Owner

- Condition: Issue is with management itself
1. Step 1: Student selects category: "Hostel Staff" or "Owner/Warden"
 2. Step 2: System routes to HMS Core admin, not to warden
 3. Step 3: Platform team reviews complaint
 4. Step 4: Independent investigation if serious
 5. Step 5: Student identity protected

AC4: Escalate Unresolved Complaint

- Condition: No response from warden after 48 hours
1. Step 1: Student sees "Escalate" button after 48 hrs
 2. Step 2: Taps escalate
 3. Step 3: Complaint forwarded to hostel owner
 4. Step 4: If still no response, escalates to HMS Core team
 5. Step 5: Platform team follows up directly

AC5: Add Update to Existing Complaint

- Condition: Issue persists or new information needed
1. Step 1: Student opens complaint tracking
 2. Step 2: Taps "Add Comment"
 3. Step 3: Types update: "AC stopped again after 2 hours"
 4. Step 4: Can attach new photos
 5. Step 5: Warden receives update notification
 6. Step 6: Complaint reopened if was marked resolved

Exception Courses:

EX1: Image Upload Failure

- Condition: Network issue prevents photo upload
1. Step 1: Student attaches 2 photos
 2. Step 2: Upload fails for 1 photo
 3. Step 3: Error: "1 photo failed to upload"
 4. Step 4: Option: "Continue without this photo" or "Retry"
 5. Step 5: Complaint can proceed with 1 photo
 6. Step 6: Can add photos later via "Add Comment"

EX2: Complaint Submission Timeout

- Condition: Server delay or network issue
1. Step 1: Student taps "Submit Complaint"
 2. Step 2: Request times out after 30 seconds
 3. Step 3: Error: "Submission failed. Please try again."
 4. Step 4: Complaint draft auto-saved locally
 5. Step 5: Student can retry without re-entering data

EX3: AI Miscategorization

- Condition: AI incorrectly categorizes complaint
1. Step 1: Student submits food quality issue
 2. Step 2: AI miscategorizes as "Maintenance"
 3. Step 3: Routed to wrong person
 4. Step 4: Warden sees mismatch, re-categorizes
 5. Step 5: Routes to correct person (cook/food manager)
 6. Step 6: Student notified of re-routing

EX4: Duplicate Complaint Detection

- Condition: Student submits same issue twice

1. Step 1: Student tries to submit AC complaint
2. Step 2: System detects similar open complaint from same room
3. Step 3: Warning: "Similar complaint already exists (#CMP2026015-042)"
4. Step 4: Options: "View existing complaint" or "Submit anyway"
5. Step 5: Prevents spam, encourages using existing complaint thread

Postconditions:

Success:

- Complaint recorded in database with unique ID
- AI categorization and routing completed
- Notifications sent to appropriate authorities
- Student can track complaint status in real-time
- Timeline created for transparency
- Resolution tracked with timestamps
- Feedback collected post-resolution
- Analytics logged for hostel quality monitoring
- Complaint history maintained for student and hostel

Failure:

- Draft saved locally for retry
- Error message shown with retry option
- Support team notified if repeated failures
- Student can contact support directly

USE CASE 13: Interact with AI Chatbot for Support

Element: Detail

Name: Get Instant Help from AI Chatbot

ID: UC-STU-013

Description: The student interacts with an AI-powered chatbot to get instant answers to common questions about hostels, bookings, payments, policies, app features, and general queries. The chatbot uses natural

language processing to understand questions, provides relevant answers, can search hostel database, and escalates to human support when needed.

Actors: Student (Primary), AI Chatbot Engine (Primary), Knowledge Base (Secondary), Human Support Agent (Secondary - escalation), Backend Services (Secondary)

Organization Benefits:

- Reduces load on human support team
- Provides 24/7 instant support
- Scales support without linear cost increase
- Captures common questions for knowledge base improvement
- Improves user experience through immediate responses
- Guides users through complex processes
- Reduces support ticket volume
- Provides data on user pain points

Preconditions:

- Student must have app open
- Internet connectivity required
- AI chatbot service operational
- Knowledge base populated and current
- Student can be logged in or guest user

Triggers:

- Student taps "Help" or chat icon in app
- Student taps "Chat with Support"
- Automated prompt: "Need help? Ask me anything!"
- Context-sensitive help: "Having trouble? I can help"

Main Course:

1. Step 1: Student confused about booking process, taps chat icon () in bottom navigation
2. Step 2: Chat interface opens with welcoming message:

HMS SUPPORT BOT 🤖

Bot: Hi Arjun! 🙋 I'm your HMS assistant.
How can I help you today?

Quick options:

- [🔍 Find Hostels]
- [📅 Booking Help]
- [💰 Payment Issues]
- [📄 My Bookings]
- [? Other Questions]

Or type your question below...

[Text input field]

3. Step 3: Student types: "How do I book a hostel?"
4. Step 4: Bot processes question using NLP
5. Step 5: Bot responds:

Bot: Great question! Booking a hostel is easy. Here's how: 📄

- 1 Search for hostels (by location or university)
- 2 Browse results and compare options
- 3 Select your preferred hostel
- 4 Choose room type and bed
- 5 Upload required documents
- 6 Review terms and make payment
- 7 Get confirmation!

[📺 Watch Video Guide]

[📖 Read Detailed Steps]

Need help with any specific step?

6. Step 6: Student asks follow-up: "What documents do I need?"
7. Step 7: Bot understands context (booking process) and responds:

Bot: For booking, you'll need: 📄

- ✅ Photo ID (Aadhaar/Passport/DL)
- ✅ Student ID Card
- ✅ Passport-size photo
- ℹ️ Parent ID (optional but recommended)

You can upload them during booking or take photos directly through the app!

💡 Tip: Keep scanned copies ready on your phone for faster booking.

[Start Booking Process]

Anything else I can help with?

8. Step 8: Student taps "Start Booking Process" button
9. Step 9: Bot redirects to search screen
10. Step 10: (Later) Student returns to chat with different question:
 - "Can I get a refund if I cancel?"
11. Step 11: Bot searches knowledge base for cancellation policy:

Bot: Refund policies depend on when you cancel: 🇳🇵

● Before join date (7+ days):
Full refund (minus platform fee)

● Before join date (3-6 days):
50% refund

● Before join date (1-2 days):
No refund (owner discretion)

● After moving in:
No refund for current month
Must give 1-month notice
Deposit refunded after inspection

Note: Individual hostels may have different policies. Check your booking confirmation.

[\[View My Booking Details\]](#)

[\[Contact Hostel Owner\]](#)

Do you have an active booking you want to cancel?

12. Step 12: Student: "Yes, I want to cancel my booking"

13. Step 13: Bot retrieves student's bookings from database:

Bot: I can help you with that. You have:

📍 Sunshine Boys Hostel
Room 301, Bed A
Join Date: Jan 20, 2026 (5 days away)
Status: Confirmed

⚠️ If you cancel now (5 days before), you'll
get 50% refund = ₹3,083.50

Are you sure you want to proceed?

[Yes, Cancel Booking]

[No, Keep Booking]

[Talk to Human Agent]

May I know why you're canceling? This helps
us improve. (optional)

14. Step 14: Student reconsiders, taps "No, Keep Booking"

15. Step 15: Bot responds:

Bot: Great! Your booking is safe. 😊

If you were having doubts, here are some
things that might help:

- [Chat with Current Residents]
- [Virtual Tour of Room]
- [Read More Reviews]
- [Contact Hostel Owner]

Anything else I can help you with?

16. Step 16: Student asks complex question: "Which hostel near IIT Delhi has best WiFi under 10k?"

17. Step 17: Bot performs intelligent search:

Bot: Let me find the best options for you... 🔍

[Searching hostels near IIT Delhi with fast
WiFi under ₹10,000...] (loading animation)

18. Step 18: Bot queries backend with parameters:

- Location: IIT Delhi
- Max price: ₹10,000
- Sort by: WiFi speed rating

19. Step 19: Bot presents results:

Bot: Found 3 hostels with excellent WiFi! 📶

1 Tech Hub Hostel

₹9,800/month

WiFi: 100 Mbps (Fiber) ⚡

Distance: 1.5 km from IIT Delhi

Rating: 4.4★

[View Details]

2 Scholar's Den

₹9,500/month

WiFi: 75 Mbps

Distance: 0.8 km from IIT Delhi

Rating: 4.5★

[View Details]

3 Campus Connect PG

₹9,200/month

WiFi: 50 Mbps

Distance: 2.0 km from IIT Delhi

Rating: 4.2★

[View Details]

Would you like me to compare these hostels?

[Compare All 3] [Refine Search]

20. Step 20: Student satisfied with results, taps "View Details" on Scholar's Den

21. Step 21: Bot redirects to hostel detail page

22. Step 22: Chat history saved for future reference

Alternate Course:

AC1: Escalate to Human Support

- Condition: Bot can't answer or student requests human
1. Step 1: Student asks complex/unusual question
 2. Step 2: Bot tries to answer but uncertain
 3. Step 3: Bot offers: "I'm not 100% sure. Would you like to speak with a human agent?"
 4. Step 4: Student taps "Yes, connect me"
 5. Step 5: Bot: "Connecting you to support team... Please hold."
 6. Step 6: Human agent takes over chat
 7. Step 7: Agent has full context of previous conversation
 8. Step 8: Agent resolves issue and closes chat

AC2: Voice Input for Questions

- Condition: Student prefers speaking over typing
1. Step 1: Student taps microphone icon in chat
 2. Step 2: Voice input activated: "Listening..."
 3. Step 3: Student speaks: "Show me girls hostels near Delhi University"
 4. Step 4: Speech-to-text conversion
 5. Step 5: Bot processes as text query
 6. Step 6: Responds with search results

AC3: Multi-language Support

- Condition: Student prefers Hindi or regional language
1. Step 1: Student types in Hindi: "मुझे हॉस्टल कैसे बुक करना है?"
 2. Step 2: Bot detects Hindi language
 3. Step 3: Responds in Hindi with instructions
 4. Step 4: Can switch between languages mid-conversation

AC4: Proactive Help Based on Context

- Condition: Bot detects student stuck on a screen
1. Step 1: Student on payment screen for 3+ minutes
 2. Step 2: Bot proactively appears: "Need help with payment? I'm here!"
 3. Step 3: Student clicks chat bubble
 4. Step 4: Bot offers payment-specific help
 5. Step 5: Guides through payment process

AC5: Quick Actions via Bot

- Condition: Bot can perform actions, not just answer
1. Step 1: Student: "Track my complaint"
 2. Step 2: Bot retrieves complaint status from database
 3. Step 3: Shows timeline directly in chat
 4. Step 4: Offers action buttons: "Add Comment", "Escalate"
 5. Step 5: Student can act without leaving chat

Exception Courses:

EX1: Bot Doesn't Understand Question

- Condition: Query is ambiguous or out of scope
1. Step 1: Student: "The thing is not working"
 2. Step 2: Bot: "I'd like to help! Can you be more specific? What's not working?"
 3. Step 3: Student clarifies
 4. Step 4: Bot provides relevant answer
 5. Step 5: If still unclear, offers human support

EX2: Chatbot Service Down

- Condition: AI service unavailable
1. Step 1: Student opens chat
 2. Step 2: Error: "Chat temporarily unavailable"
 3. Step 3: Fallback options shown:

- Browse FAQs
- Email support
- Call support hotline

4. Step 4: Human support notified of bot downtime

EX3: Outdated Information Given

- Condition: Knowledge base not updated
1. Step 1: Bot provides old policy information
 2. Step 2: Student: "But the website says something different"
 3. Step 3: Bot: "Thanks for catching that! Let me verify..."
 4. Step 4: Connects to human agent for clarification
 5. Step 5: Logs discrepancy for knowledge base update

EX4: Network Loss During Chat

- Condition: Internet disconnects mid-conversation
1. Step 1: Student asking question
 2. Step 2: Network drops
 3. Step 3: Chat shows offline indicator
 4. Step 4: Messages queued locally
 5. Step 5: When reconnected, conversation resumes
 6. Step 6: Queued messages sent automatically

Postconditions:

Success:

- Student receives instant answer to question
- Issue resolved without human intervention (if possible)
- Student can continue task/journey
- Conversation logged for analytics
- Common questions identified for FAQ improvement
- Escalated to human if needed
- Student satisfaction improved through quick help

Failure:

- Student directed to alternative support channels
- Human support notified if bot can't help
- Error logged for service improvement
- FAQ fallback provided

USE CASE 14: Send Emergency Alert

Element: Detail

Name: Send Emergency Alert for Immediate Help

ID: UC-STU-014

Description: The student activates emergency alert system when facing urgent safety or health emergency at hostel. The system immediately notifies hostel warden, owner, emergency contacts (parents), nearby students, and optionally local emergency services with student's location and nature of emergency.

Actors: Student (Primary), Emergency Alert System (Primary), Warden (Secondary), Hostel Owner (Secondary), Parent/Guardian (Secondary), Nearby Students (Secondary), Emergency Services (Secondary - optional), Location Services (Secondary)

Organization Benefits:

- Ensures student safety through rapid response
- Builds trust in platform's commitment to safety
- Provides legal protection through documented alerts
- Enables quick coordination in emergencies
- Reduces response time significantly
- Creates accountability for hostel safety
- Provides peace of mind for parents
- Differentiates platform as safety-focused

Preconditions:

- Student must be logged in
- Student must have active booking/stay at hostel
- Emergency contacts must be configured in profile
- GPS/location services should be enabled
- Internet or SMS capability required
- Warden and owner contact details must be available

Triggers:

- Student taps "Emergency" button in app
- Student uses quick emergency gesture (e.g., shake phone 3 times)

- Critical safety situation requiring immediate help

Main Course:

1. Step 1: Student in Room 301 faces medical emergency (severe chest pain)
2. Step 2: Student unlocks phone in distress
3. Step 3: Opens HMS Core app
4. Step 4: Taps prominent "🚨 Emergency" button (red, always visible on dashboard)
5. Step 5: Emergency alert screen appears immediately:

🚨 EMERGENCY ALERT

What type of emergency?

[🏥 MEDICAL]

Health issue, injury, illness

[⚠️ SAFETY THREAT]

Danger, violence, intrusion

[🔥 FIRE/ACCIDENT]

Fire, gas leak, structural issue

[❓ OTHER EMERGENCY]

Other urgent situation

[Cancel] (5 sec countdown)

6. Step 6: Student taps "🏥 MEDICAL"
7. Step 7: Confirmation screen with auto-countdown:

SENDING MEDICAL EMERGENCY ALERT

Your location and details will be shared with:

- ✓ Hostel Warden
- ✓ Hostel Owner
- ✓ Your Emergency Contacts (Parents)
- ✓ Students in your hostel

Sending in 3... 2... 1...

[SEND NOW] [Cancel]

8. Step 8: Student taps "SEND NOW" (or auto-sends after 3 seconds)

9. Step 9: System immediately activates emergency protocol:

- A. Location Capture:
 - GPS coordinates captured
 - Address: Sunshine Boys Hostel, Room 301
 - Timestamp: Jan 15, 2026, 11:45 PM
- B. Student Info Package:
 - Name: Arjun Singh
 - Age: 21
 - Blood Group: O+ (from profile)
 - Known Allergies: None (from profile)
 - Emergency Type: Medical
 - Room: 301, Bed A

10. Step 10: Simultaneous Multi-Channel Notifications Sent:

- To Warden (Mr. Gupta):

 MEDICAL EMERGENCY ALERT

Student: Arjun Singh (21, Room 301)

Emergency: Medical

Time: 11:45 PM

Blood Group: O+

Location: Room 301, Floor 3

[View on Map] [Call Student: +91 98765 43210]

Sent via: Push + SMS + Call

- To Hostel Owner:

- Same alert via push notification and SMS

- To Parents (Emergency Contacts):

 EMERGENCY ALERT - Your son needs help

Arjun has activated a medical emergency alert
from Sunshine Boys Hostel, Room 301.

Time: 11:45 PM, Jan 15, 2026

Hostel staff has been notified.

Warden: Mr. Gupta (+91 98765 00000)

Location: [Map Link]

Sent via: SMS + Call + App Notification

- To Nearby Students (Same Hostel):

 Emergency in your hostel

A student in Room 301 needs immediate help.
Medical emergency reported.

If you're nearby and can assist, please help
or alert warden immediately.

Warden has been notified.

- Automated Voice Call to Warden:
- System places automated call
- Pre-recorded message: "Emergency alert from Room 301..."
- Press 1 to acknowledge, Press 2 for details

11. Step 11: Student's screen shows:

 EMERGENCY ALERT SENT

Help is on the way!

Notifications sent to:

- ✓ Warden (Mr. Gupta)
- ✓ Hostel Owner
- ✓ Your Parents
- ✓ Nearby Students

Emergency ID: #EMG2026015-001

Time: 11:45 PM

WHAT TO DO NOW:

- Stay calm and in your current location
- Keep your phone nearby
- Warden has been alerted and should respond soon
- Medical help is being arranged

[Call Warden Now: +91 98765 00000]

[Call Emergency Services: 112]

[Update Status]

[Cancel Alert (if resolved)]

12. Step 12: Warden receives all notifications within seconds
13. Step 13: Warden acknowledges by pressing 1 on call or tapping app notification
14. Step 14: Student's screen updates:

Warden Acknowledged: 11:46 PM

Status: Help on the way 🚑

Mr. Gupta is responding to your emergency

15. Step 15: Warden arrives at Room 301 within 3 minutes
16. Step 16: Warden assesses situation, calls ambulance if needed
17. Step 17: Warden updates alert status in app: "Student being taken to hospital"
18. Step 18: Parents receive real-time updates:
 - "Warden responded. Student being assisted."
 - "Ambulance called. Taking to City Hospital."
19. Step 19: Once student safe, warden marks emergency as "Resolved"
20. Step 20: Student/Parents receive final notification:

Emergency Resolved

Your emergency has been addressed.

Student is safe and receiving medical attention.

Hospital: City Hospital, North Campus

Contact: Dr. Sharma (+91 98765 11111)

Emergency Log saved for reference.

[\[View Complete Timeline\]](#)

[\[Contact Warden\]](#)

21. Step 21: Complete emergency timeline logged:
 - 11:45 PM: Alert activated
 - 11:45 PM: Notifications sent
 - 11:46 PM: Warden acknowledged
 - 11:48 PM: Warden arrived at room

- 11:52 PM: Ambulance called
- 12:05 AM: Student transported to hospital
- 12:30 AM: Emergency resolved

22. Step 22: Platform admin receives emergency report for quality/safety monitoring

Alternate Course:

AC1: Safety Threat Emergency

- Condition: Student faces danger or intrusion
1. Step 1: Student selects " ⚠ SAFETY THREAT"
 2. Step 2: Alert sent immediately (no countdown)
 3. Step 3: Additional notification to local police (optional, based on settings)
 4. Step 4: Nearby students alerted: "Safety emergency in building"
 5. Step 5: Campus security notified if integrated
 6. Step 6: Silent mode option: Phone vibrates but no sound

AC2: Fire/Accident Emergency

- Condition: Fire or structural emergency in hostel
1. Step 1: Student selects " 🔥 FIRE/ACCIDENT"
 2. Step 2: Alert sent to all hostel residents
 3. Step 3: Evacuation instructions shown
 4. Step 4: Fire department auto-notified (if integration available)
 5. Step 5: Warden receives evacuation protocol checklist
 6. Step 6: All residents receive: "Fire emergency. Evacuate immediately."

AC3: False Alarm / Accidental Activation

- Condition: Student activates emergency by mistake
1. Step 1: Student realizes mistake within 5-second countdown
 2. Step 2: Taps "Cancel" before auto-send
 3. Step 3: Alert canceled, no notifications sent
 4. Step 4: Confirmation: "Emergency alert canceled"
 5. Step 5: If already sent, student can tap "Cancel Alert"

6. Step 6: "False alarm" notification sent to all recipients
7. Step 7: Logged as false alarm (to track accidental activations)

AC4: Quick Emergency Gesture

- Condition: Student can't access app normally
1. Step 1: Student shakes phone vigorously 3 times
 2. Step 2: Emergency alert activates even from locked screen
 3. Step 3: Vibration confirms activation
 4. Step 4: Alert type defaults to "General Emergency"
 5. Step 5: All notifications sent automatically
 6. Step 6: Provides fastest emergency response

AC5: No Response from Warden

- Condition: Warden doesn't acknowledge within 2 minutes
1. Step 1: Alert sent at 11:45 PM
 2. Step 2: No acknowledgment by 11:47 PM
 3. Step 3: System auto-escalates
 4. Step 4: Second alert sent to owner
 5. Step 5: Direct call placed to warden (automated)
 6. Step 6: HMS Core emergency team notified
 7. Step 7: Parent receives update: "Escalating - no initial response"

Exception Courses:

EX1: No Internet Connection

- Condition: Student offline when activating emergency
1. Step 1: Student taps emergency button
 2. Step 2: System detects no internet
 3. Step 3: Falls back to SMS-based alerts
 4. Step 4: Sends emergency SMS to warden + parents
 5. Step 5: Alert queued for sending when online

6. Step 6: Shows: "Offline - sending via SMS"

EX2: GPS Location Unavailable

- Condition: GPS off or not working
1. Step 1: System tries to capture location
 2. Step 2: GPS unavailable
 3. Step 3: Uses last known location
 4. Step 4: Shows: "Location: Room 301 (from profile)"
 5. Step 5: Alert sent with profile-based location
 6. Step 6: Note added: "GPS location unavailable"

EX3: Emergency Contacts Not Configured

- Condition: Student hasn't added emergency contacts
1. Step 1: Emergency activated
 2. Step 2: System detects no emergency contacts
 3. Step 3: Alert still sent to warden/owner
 4. Step 4: Warning shown: "No emergency contacts configured"
 5. Step 5: After emergency, prompts: "Add emergency contacts now"

EX4: Multiple Emergencies Simultaneously

- Condition: Multiple students in same hostel activate alerts
1. Step 1: 3 students activate emergency within 1 minute
 2. Step 2: Warden receives consolidated alert:
 - "3 EMERGENCIES in your hostel"
 3. Step 3: List shows all emergencies with priority
 4. Step 4: Warden can acknowledge each separately
 5. Step 5: System suggests calling for backup help
 6. Step 6: Owner and admin auto-notified of mass emergency

EX5: Student Becomes Unresponsive

- Condition: Student can't update status after sending alert

1. Step 1: Alert sent but no further updates from student
2. Step 2: After 10 minutes, system sends reminder to warden
3. Step 3: "No update from student - please verify situation"
4. Step 4: Parents notified: "Warden investigating"
5. Step 5: Platform tracks time to resolution
6. Step 6: Escalation if no resolution within 30 minutes

Postconditions:

Success:

- Emergency alert sent to all relevant parties within seconds
- Student's location and information shared accurately
- Warden/staff responds promptly to emergency
- Parents kept informed in real-time
- Complete timeline logged for documentation
- Emergency resolved with appropriate action taken
- Student safety ensured
- Platform demonstrates commitment to student welfare
- Analytics logged for safety monitoring

Failure:

- Offline fallback mechanisms activated
- Alternative contact methods used (SMS, calls)
- Escalation protocols triggered automatically
- Platform admin notified of system failure
- Manual intervention procedures initiated
- Emergency still addressed through backup channels